

Slip 1:-

Q.1) Write a C Menu driven Program to implement following functionality

- Accept Available
- Display Allocation, Max
- Display the contents of need matrix
- Display Available

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	2	3	2	9	7	5	3	3	2
P1	4	0	0	5	2	2			
P2	5	0	4	1	0	4			
P3	4	3	3	4	4	4			
P4	2	2	4	6	5	5			

[15 Marks]

Q.2 Write a C program that behaves like a shell which displays the command

prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'count' as

- \$ count c filename: It will display the number of characters in given file
- \$ count w filename: It will display the number of words in given file
- \$ count l filename: It will display the number of lines in given file

[15 Marks]

Q.3. Oral/Viva

[5 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
int alloc[5][3] = {
```

```
{2,3,2},
```

```
{4,0,0},
```

```
{5,0,4},
```

```
{4,3,3},
```

```
{2,2,4}
```

```
};
```

```
int max[5][3] = {
```

```
    {9,7,5},
```

```
    {5,2,2},
```

```
    {1,0,4},
```

```
    {4,4,4},
```

```
    {6,5,5}
```

```
};
```

```
int avail[3];
```

```
void accept_available()
```

```
{
```

```
    printf("\nEnter Available resources (A B C): ");
```

```
    scanf("%d%d%d",&avail[0],&avail[1],&avail[2]);
```

```
}
```

```
void display_alloc_max()
```

```
{
```

```
    int i;
```

```
    printf("\nProcess\tAllocation\tMax\n");
```

```
    printf("\tA B C\t\tA B C\n");
```

```
    for(i=0;i<5;i++)
```

```
        printf("P%d\t%d %d %d\t\t%d %d %d\n",
```

```
        i,alloc[i][0],alloc[i][1],alloc[i][2],
```

```
        max[i][0],max[i][1],max[i][2]);
```

```
}
```

```
void display_need()
```

```
{
```

```
    int i,j;
```

```
    printf("\nNeed Matrix\n");
```

```
    printf("Process\tA\tB\tC\n");
```

```
    for(i=0;i<5;i++)
```

```
    {
```

```
        printf("P%d\t",i);
```

```
        for(j=0;j<3;j++)
```

```
            printf("%d\t",max[i][j]-alloc[i][j]);
```

```
        printf("\n");
```

```
    }
```

```
}
```

```
void display_available()
```

```
{
```

```
    printf("\nAvailable\n");
```

```
    printf("A\tB\tC\n");
```

```
    printf("%d\t%d\t%d\n",avail[0],avail[1],avail[2]);
```

```
}
```

```
int main()
```

```
{
```

```
int ch;

do
{
    printf("\n\n--- BANKER'S ALGORITHM ---");
    printf("\n1. Accept Available");
    printf("\n2. Display Allocation, Max");
    printf("\n3. Display Need Matrix");
    printf("\n4. Display Available");
    printf("\n5. Exit");

    printf("\nEnter choice: ");
    scanf("%d",&ch);

    switch(ch)
    {
        case 1:
            accept_available();
            break;

        case 2:
            display_alloc_max();
            break;

        case 3:
            display_need();
            break;

        case 4:
```

```
        display_available();
        break;

    case 5:
        exit(0);

    default:
        printf("\nInvalid choice!");
    }

}while(1);

return 0;
}
```

Q2)

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/wait.h>

char comm[100], *args[10];
int tot;

void getcomm()
{
    int len;
```

```
fgets(comm,80,stdin);  
len=strlen(comm);  
comm[len-1]='\0';  
}
```

```
void sep_args()  
{  
    char *token;  
    tot=0;  
  
    token=strtok(comm," ");  
  
    while(token!=NULL)  
    {  
        args[tot]=malloc(20);  
        strcpy(args[tot++],token);  
        token=strtok(NULL," ");  
    }  
  
    args[tot]=NULL;  
}
```

```
void count(char option[], char fname[])  
{  
    FILE *fp;  
    int ch, characters=0, words=0, lines=0, inword=0;  
  
    fp=fopen(fname,"r");
```

```
if(fp==NULL)
{
    printf("Unable to open file\n");
    return;
}

while((ch=fgetc(fp))!=EOF)
{
    characters++;

    if(ch=='\n')
        lines++;

    if(ch==' ' || ch=='\t' || ch=='\n')
        inword=0;
    else if(inword==0)
    {
        words++;
        inword=1;
    }
}

fclose(fp);

if(strcmp(option,"c")==0)
    printf("Total characters = %d\n",characters);
else if(strcmp(option,"w")==0)
    printf("Total words = %d\n",words);
else if(strcmp(option,"l")==0)
```

```
        printf("Total lines = %d\n",lines);
    else
        printf("Invalid option\n");
}
```

```
int main()
{
    pid_t pid;
    int status;
    char str[100];

    while(1)
    {
        printf("$ ");
        getcomm();
        sep_args();

        if(args[0]==NULL)
            continue;

        /* count command */
        if(strcmp(args[0],"count")==0)
        {
            if(tot!=3)
            {
                printf("Usage: count c|w|l filename\n");
                continue;
            }
        }
    }
}
```

```
        count(args[1],args[2]);
    }
    else
    {
        pid=fork();

        if(pid<0)
        {
            printf("Fork Failed\n");
            continue;
        }

        if(pid==0)
        {
            execvp(args[0],args);

            strcpy(str,"./");
            strcat(str,args[0]);
            execvp(str,args);

            printf("%s : Command not found\n",args[0]);
            exit(0);
        }
        else
            waitpid(pid,&status,0);
    }
    return 0;
}
```

Slip 2:-

Q.1 Write a program to simulate Linked file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- Show Bit Vector
- Create New File
- Show Directory
- Exit

[15 Marks]

Q.2 Write a simulation program for disk scheduling using FCFS algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.
55, 58, 39, 18, 90, 160, 150, 38, 184
Start Head Position: 50

[15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<time.h>
```

```
struct file
```

```
{
```

```
    char name[20];
```

```
    int start;
```

```
    int last;
```

```
} dir[20];
```

```
int bit[100], next[100];
```

```
int n, fcount=0;
```

```
void showBitVector()
{
    int i;

    printf("\nBit Vector:\n");

    for(i=0;i<n;i++)
        printf("%d ",bit[i]);

    printf("\n");
}
```

```
void createFile()
{
    char name[20];
    int blocks, i, j, count=0;
    int first=-1, last=-1;

    printf("\nEnter file name: ");
    scanf("%s",name);

    printf("Enter number of blocks: ");
    scanf("%d",&blocks);

    for(i=0;i<n;i++)
        if(bit[i]==1)
            count++;

    if(count<blocks)
```

```
{  
    printf("\nNot enough free blocks!\n");  
    return;  
}
```

```
for(i=0;i<n && blocks>0;i++)
```

```
{  
    if(bit[i]==1)  
    {  
        bit[i]=0;  
  
        if(first==-1)  
            first=i;  
        else  
            next[last]=i;  
  
        last=i;  
        blocks--;  
    }  
}
```

```
next[last]=-1;
```

```
for(j=0;j<fcount;j++)
```

```
{  
    if(strcmp(dir[j].name,name)==0)  
    {  
        printf("\nFile already exists!\n");  
        return;  
    }
```

```

    }
}

strcpy(dir[fcount].name,name);
dir[fcount].start=first;
dir[fcount].last=last;
fcount++;

printf("\nFile created successfully.");
printf("\nStarting block = %d",first);
printf("\nLast block = %d\n",last);
}

void showDirectory()
{
    int i, j;

    printf("\nDirectory:\n");
    printf("File\tStart\tBlocks\n");

    for(i=0;i<fcount;i++)
    {
        printf("%s\t%d\t",dir[i].name,dir[i].start);

        j=dir[i].start;

        while(j!=-1)
        {
            printf("%d ",j);

```

```

        j=next[j];
    }

    printf("\n");
}

int main()
{
    int i,ch;

    printf("Enter number of disk blocks: ");
    scanf("%d",&n);

    srand(time(NULL));

    for(i=0;i<n;i++)
    {
        bit[i]=rand()%2;
        next[i]=-1;
    }

    do
    {
        printf("\n\n--- LINKED FILE ALLOCATION ---");
        printf("\n1. Show Bit Vector");
        printf("\n2. Create New File");
        printf("\n3. Show Directory");
        printf("\n4. Exit");
    }

```

```
printf("\nEnter choice: ");  
scanf("%d",&ch);  
  
switch(ch)  
{  
    case 1:  
        showBitVector();  
        break;  
  
    case 2:  
        createFile();  
        break;  
  
    case 3:  
        showDirectory();  
        break;  
  
    case 4:  
        exit(0);  
  
    default:  
        printf("\nInvalid choice!");  
}  
  
}while(1);  
  
return 0;  
}
```

Q2)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
int main()
```

```
{
```

```
    int n,i,head,total=0;
```

```
    int req[100];
```

```
    printf("Enter total number of disk blocks: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter number of disk requests: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter disk request string:\n");
```

```
    for(i=0;i<n;i++)
```

```
        scanf("%d",&req[i]);
```

```
    printf("\nEnter current head position: ");
```

```
    scanf("%d",&head);
```

```
    printf("\nFCFS Disk Scheduling\n");
```

```
    printf("Request order:\n");
```

```
    printf("%d",head);
```

```
for(i=0;i<n;i++)
{
    total=total+abs(head-req[i]);
    head=req[i];

    printf(" -> %d",req[i]);
}

printf("\n\nTotal head movements = %d\n",total);

return 0;
}
```

Slip 3:-

Q.1 Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following snapshot of system, A, B, C and D is the resource type.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

- a) Calculate and display the content of need matrix?
b) Is the system in safe state? If display the safe sequence.

[15 Marks]

Q.2 Write a simulation program for disk scheduling using SSTF algorithm.

Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

98, 183, 37, 122, 14, 124, 65

Start Head Position: 53

[15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
int alloc[5][4] = {
```

```
    {0,0,1,2},
```

```
    {1,0,0,0},
```

```
    {1,3,5,4},
```

```
    {0,6,3,2},
```

```
    {0,0,1,4}
```

```
};
```

```
int max[5][4] = {
```

```
    {0,0,1,2},
```

```

    {1,7,5,0},
    {2,3,5,6},
    {0,6,5,2},
    {0,6,5,6}
};

int avail[4] = {1,5,2,0};
int need[5][4];

void calculateNeed()
{
    int i,j;

    for(i=0;i<5;i++)
        for(j=0;j<4;j++)
            need[i][j]=max[i][j]-alloc[i][j];
}

void displayNeed()
{
    int i,j;

    printf("\nNeed Matrix\n");
    printf("Process\tA\tB\tC\tD\n");

    for(i=0;i<5;i++)
    {
        printf("P%d\t",i);

```

}

```

        {
            for(j=0;j<4;j++)
                work[j]+=alloc[i][j];

            safe[count++]=i;
            finish[i]=1;
            found=1;
        }
    }
}

if(found==0)
    break;
}

if(count==5)
{
    printf("\nSystem is in SAFE state.");
    printf("\nSafe Sequence: ");

    for(i=0;i<5;i++)
        printf("P%d ",safe[i]);

    printf("\n");
}
else
{
    printf("\nSystem is NOT in a safe state.\n");
}

```

```
}
```

```
int main()
```

```
{
```

```
    calculateNeed();
```

```
    printf("Available Resources: ");
```

```
    printf("%d %d %d %d\n",avail[0],avail[1],avail[2],avail[3]);
```

```
    displayNeed();
```

```
    safety();
```

```
    return 0;
```

```
}
```

Q2)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
int main()
```

```
{
```

```
    int blocks,n,i,j,head,total=0;
```

```
    int req[100],done[100],min,pos,diff;
```

```
    printf("Enter total number of disk blocks: ");
```

```
    scanf("%d",&blocks);
```

```
printf("Enter number of disk requests: ");  
scanf("%d",&n);
```

```
printf("Enter disk request string:\n");
```

```
for(i=0;i<n;i++)  
{  
    scanf("%d",&req[i]);  
    done[i]=0;  
}
```

```
printf("Enter current head position: ");  
scanf("%d",&head);
```

```
printf("\nSSTF Request Order:\n");  
printf("%d",head);
```

```
for(i=0;i<n;i++)  
{  
    min=9999;  
    pos=-1;  
  
    for(j=0;j<n;j++)  
    {  
        if(done[j]==0)  
        {  
            diff=abs(head-req[j]);  
  
            if(diff<min)
```

```
        {
            min=diff;
            pos=j;
        }
    }

    total=total+min;
    head=req[pos];
    done[pos]=1;

    printf(" -> %d",head);
}

printf("\n\nTotal Head Movements = %d\n",total);

return 0;
}
```

Slip 4:-

Q.1 Implement the Menu driven Banker's algorithm for accepting Allocation, Max from user

- a) Accept Available
- b) Display Allocation, Max
- c) Find Need and display It,
- d) Display Available

Consider the system with 3 resources types A, B, and C with 7, 2, 6 instances respectively consider the following snapshot:

Process	Allocation			Request		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	4	0	0	5	2	2
P2	5	0	4	1	0	4
P3	4	3	3	4	4	4
P4	2	2	4	6	5	5

[15 Marks]

Q.2 Write a simulation program for disk scheduling using SCAN algorithm.

Accept total Number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.
82, 170, 43, 140, 24, 16, 190, 65

Starting Head position= 50

Direction: Left

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
int alloc[5][3], max[5][3], need[5][3];
```

```
int avail[3];
```

```
void accept()
```

```
{
```

```
    int i,j;
```

```
    printf("\nEnter Allocation Matrix:\n");
```

```
for(i=0;i<5;i++)
{
    printf("P%d: ",i);
    for(j=0;j<3;j++)
        scanf("%d",&alloc[i][j]);
}
```

```
printf("\nEnter Max Matrix:\n");
```

```
for(i=0;i<5;i++)
{
    printf("P%d: ",i);
    for(j=0;j<3;j++)
        scanf("%d",&max[i][j]);
}
```

```
printf("\nEnter Available (A B C): ");
scanf("%d%d%d",&avail[0],&avail[1],&avail[2]);
```

```
for(i=0;i<5;i++)
    for(j=0;j<3;j++)
        need[i][j]=max[i][j]-alloc[i][j];
```

```
printf("\nData accepted successfully.");
}
```

```
void displayAllocMax()
{
    int i;
```

```
printf("\nProcess\tAllocation\tMax\n");
```

```
printf("\tA B C\t\tA B C\n");
```

```
for(i=0;i<5;i++)
```

```
    printf("P%d\t%d %d %d\t\t%d %d %d\n",
```

```
    i,
```

```
    alloc[i][0],alloc[i][1],alloc[i][2],
```

```
    max[i][0],max[i][1],max[i][2]);
```

```
}
```

```
void displayNeed()
```

```
{
```

```
    int i;
```

```
    printf("\nNeed Matrix\n");
```

```
    printf("Process\tA\tB\tC\n");
```

```
for(i=0;i<5;i++)
```

```
    printf("P%d\t%d\t%d\t%d\n",
```

```
    i,need[i][0],need[i][1],need[i][2]);
```

```
}
```

```
void displayAvailable()
```

```
{
```

```
    printf("\nAvailable Resources\n");
```

```
    printf("A\tB\tC\n");
```

```
    printf("%d\t%d\t%d\n",
```

```
    avail[0],avail[1],avail[2]);
```

```
}
```

```
int main()
```

```
{
```

```
    int ch;
```

```
    do
```

```
    {
```

```
        printf("\n\n--- BANKER'S ALGORITHM ---");
```

```
        printf("\n1. Accept Allocation, Max and Available");
```

```
        printf("\n2. Display Allocation, Max");
```

```
        printf("\n3. Find and Display Need");
```

```
        printf("\n4. Display Available");
```

```
        printf("\n5. Exit");
```

```
        printf("\nEnter choice: ");
```

```
        scanf("%d",&ch);
```

```
        switch(ch)
```

```
        {
```

```
            case 1:
```

```
                accept();
```

```
                break;
```

```
            case 2:
```

```
                displayAllocMax();
```

```
                break;
```

```
            case 3:
```

```

        displayNeed();
        break;

    case 4:
        displayAvailable();
        break;

    case 5:
        exit(0);

    default:
        printf("\nInvalid choice!");
    }

}while(1);

return 0;
}

```

Q2)

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

```

```

int main()
{
    int blocks,n,i,j,head,total=0;
    int req[100],done[100];
    int temp;

```

```
printf("Enter total number of disk blocks: ");  
scanf("%d",&blocks);
```

```
printf("Enter number of disk requests: ");  
scanf("%d",&n);
```

```
printf("Enter disk request string:\n");
```

```
for(i=0;i<n;i++)
```

```
{  
    scanf("%d",&req[i]);  
    done[i]=0;
```

```
}
```

```
printf("Enter current head position: ");
```

```
scanf("%d",&head);
```

```
/* Sort requests */
```

```
for(i=0;i<n-1;i++)
```

```
    for(j=i+1;j<n;j++)
```

```
        if(req[i]>req[j])
```

```
        {
```

```
            temp=req[i];
```

```
            req[i]=req[j];
```

```
            req[j]=temp;
```

```
        }
```

```
printf("\nSCAN Request Order:\n");
```

```
printf("%d",head);
```

```
/* Move Left */
```

```
for(i=n-1;i>=0;i--)
```

```

{
    if(req[i]<head)
    {
        total += abs(head-req[i]);
        head=req[i];
        printf(" -> %d",head);
    }
}

/* Move to left end */
if(head!=0)
{
    total += head;
    head=0;
    printf(" -> 0");
}

/* Reverse direction and move Right */
for(i=0;i<n;i++)
{
    if(req[i]>head)
    {
        total += abs(head-req[i]);
        head=req[i];
        printf(" -> %d",head);
    }
}

printf("\n\nTotal Head Movements = %d\n",total);

return 0;
}

```

Slip 5:-

Q.1 Write the simulation program for demand paging and show the page scheduling total and number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames.

Reference String: 3,4,5,6,3,4,7,3,4,5,6,7,2,4,6

[15 Marks]

Q2 Consider a system with 'n' processes and 'm' resource types. Accept number of instances for every resource type. For each process accept the allocation and maximum requirement matrices. Write a program to display the contents of need matrix and to check if the given request of a process can be granted immediately or not. (Use resource request algorithm)

[15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
int main()
```

```
{
```

```
    int frames[20], n;
```

```
    int pages[] = {3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
```

```
    int size = 15;
```

```
    int i,j,pos=0,faults=0,found;
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
```

```
        frames[i]=-1;
```

```
    printf("\nPage\tFrames\t\tPage Fault\n");
```

```
printf("-----\n");
```

```
for(i=0;i<size;i++)
```

```
{
```

```
    found=0;
```

```
    /* Check whether page is already present */
```

```
    for(j=0;j<n;j++)
```

```
    {
```

```
        if(frames[j]==pages[i])
```

```
        {
```

```
            found=1;
```

```
            break;
```

```
        }
```

```
    }
```

```
    /* Page fault */
```

```
    if(found==0)
```

```
    {
```

```
        frames[pos]=pages[i];
```

```
        pos=(pos+1)%n;
```

```
        faults++;
```

```
        printf("%d\t",pages[i]);
```

```
    for(j=0;j<n;j++)
```

```
        printf("%d ",frames[j]);
```

```
    printf("\tYES\n");
```

```

    }
    else
    {
        printf("%d\t",pages[i]);

        for(j=0;j<n;j++)
            printf("%d ",frames[j]);

        printf("\tNO\n");
    }
}

printf("-----\n");
printf("Total Page Faults = %d\n",faults);

return 0;
}

```

Q2)

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int alloc[20][20], max[20][20], need[20][20];
int avail[20], work[20], finish[20];
int n,m;

void calculateNeed()
{
    int i,j;

```

```
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];
}
```

```
void displayNeed()
```

```
{
    int i,j;

    printf("\nNeed Matrix:\n");
    printf("Process\t");
```

```
    for(j=0;j<m;j++)
        printf("%c\t",'A'+j);
```

```
    printf("\n");
```

```
    for(i=0;i<n;i++)
```

```
    {
        printf("P%d\t",i);
```

```
        for(j=0;j<m;j++)
            printf("%d\t",need[i][j]);
```

```
        printf("\n");
```

```
    }
}
```

```

int safety()
{
    int i,j,count=0,found;

    for(j=0;j<m;j++)
        work[j]=avail[j];

    for(i=0;i<n;i++)
        finish[i]=0;

    while(count<n)
    {
        found=0;

        for(i=0;i<n;i++)
        {
            if(finish[i]==0)
            {
                for(j=0;j<m;j++)
                    if(need[i][j]>work[j])
                        break;

                if(j==m)
                {
                    for(j=0;j<m;j++)
                        work[j]+=alloc[i][j];

                    finish[i]=1;
                    count++;
                }
            }
        }
    }
}

```

```

        found=1;
    }
}

if(found==0)
    return 0;
}

return 1;
}

void resourceRequest()
{
    int p,i;
    int req[20];

    printf("\nEnter process number making request: ");
    scanf("%d",&p);

    printf("Enter request vector:\n");

    for(i=0;i<m;i++)
    {
        printf("%c: ", 'A'+i);
        scanf("%d",&req[i]);
    }

    /* Step 1: Request <= Need */

```

```
for(i=0;i<m;i++)
{
    if(req[i]>need[p][i])
    {
        printf("\nError: Process has exceeded its maximum claim.\n");
        return;
    }
}
```

/* Step 2: Request <= Available */

```
for(i=0;i<m;i++)
{
    if(req[i]>avail[i])
    {
        printf("\nResources are not available.");
        printf("\nProcess P%d must wait.\n",p);
        return;
    }
}
```

/* Step 3: Pretend to allocate resources */

```
for(i=0;i<m;i++)
{
    avail[i]-=req[i];
    alloc[p][i]+=req[i];
    need[p][i]-=req[i];
}
```

/* Step 4: Check safe state */

```

if(safety())
{
    printf("\nRequest can be GRANTED immediately.");
    printf("\nSystem remains in SAFE state.\n");
}
else
{
    /* Undo allocation */
    for(i=0;i<m;i++)
    {
        avail[i]+=req[i];
        alloc[p][i]-=req[i];
        need[p][i]+=req[i];
    }

    printf("\nRequest CANNOT be granted immediately.");
    printf("\nSystem would become UNSAFE.\n");
}
}

```

```

int main()
{
    int i,j;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter number of resource types: ");
    scanf("%d",&m);

```

```
printf("\nEnter total available instances:\n");
```

```
for(i=0;i<m;i++)  
{  
    printf("%c: ", 'A'+i);  
    scanf("%d",&avail[i]);  
}
```

```
printf("\nEnter Allocation Matrix:\n");
```

```
for(i=0;i<n;i++)  
{  
    printf("P%d: ",i);  
  
    for(j=0;j<m;j++)  
        scanf("%d",&alloc[i][j]);  
}
```

```
printf("\nEnter Maximum Requirement Matrix:\n");
```

```
for(i=0;i<n;i++)  
{  
    printf("P%d: ",i);  
  
    for(j=0;j<m;j++)  
        scanf("%d",&max[i][j]);  
}
```

```
/* Calculate currently available resources */  
for(j=0;j<m;j++)  
    for(i=0;i<n;i++)  
        avail[j]-=alloc[i][j];  
  
calculateNeed();  
displayNeed();  
  
resourceRequest();  
  
return 0;  
}
```

Slip 6:-

Q.1. Write a program to simulate Sequential file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- Show Bit Vector
- Create New File
- Show Directory
- Exit

[15 Marks]

Q.2 Write a simulation program for disk scheduling using C-SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. also display the total number of head movements (**Assume disk size=200**)
15, 45, 75, 105, 135, 165, 195, 80
Starting Head Position: 100
Direction: Right

Q1)

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<time.h>
```

```
struct file
{
    char name[20];
    int start;
    int length;
} dir[20];
```

```
int bit[100];
int n, fcount=0;
```

```
void showBitVector()
{
```

```
int i;
```

```
printf("\nBit Vector:\n");
```

```
for(i=0;i<n;i++)
```

```
    printf("%d ",bit[i]);
```

```
printf("\n");
```

```
}
```

```
void createFile()
```

```
{
```

```
    char name[20];
```

```
    int blocks,i,j,count=0;
```

```
    printf("\nEnter file name: ");
```

```
    scanf("%s",name);
```

```
    printf("Enter number of blocks: ");
```

```
    scanf("%d",&blocks);
```

```
/* Check whether file already exists */
```

```
for(i=0;i<fcount;i++)
```

```
{
```

```
    if(strcmp(dir[i].name,name)==0)
```

```
    {
```

```
        printf("\nFile already exists!\n");
```

```
        return;
```

```
    }
```

```
}
```

```
/* Find consecutive free blocks */
```

```
for(i=0;i<=n-blocks;i++)
```

```
{
```

```
    count=0;
```

```
    for(j=i;j<i+blocks;j++)
```

```
    {
```

```
        if(bit[j]==1)
```

```
            count++;
```

```
        else
```

```
            break;
```

```
    }
```

```
    if(count==blocks)
```

```
    {
```

```
        for(j=i;j<i+blocks;j++)
```

```
            bit[j]=0;
```

```
        strcpy(dir[fcount].name,name);
```

```
        dir[fcount].start=i;
```

```
        dir[fcount].length=blocks;
```

```
        fcount++;
```

```
        printf("\nFile created successfully!");
```

```
        printf("\nStarting block = %d",i);
```

```
        printf("\nLength = %d blocks\n",blocks);
```

```

        return;
    }
}

printf("\nRequired contiguous blocks are not available!\n");
}

void showDirectory()
{
    int i;

    printf("\nDirectory:\n");
    printf("File\tStart\tLength\n");

    for(i=0;i<fcount;i++)
        printf("%s\t%d\t%d\n",
            dir[i].name,
            dir[i].start,
            dir[i].length);
}

int main()
{
    int i,ch;

    printf("Enter number of disk blocks: ");
    scanf("%d",&n);

    srand(time(NULL));

```

```

/* 1 = free, 0 = allocated */
for(i=0;i<n;i++)
    bit[i]=rand()%2;

do
{
    printf("\n\n--- SEQUENTIAL FILE ALLOCATION ---");
    printf("\n1. Show Bit Vector");
    printf("\n2. Create New File");
    printf("\n3. Show Directory");
    printf("\n4. Exit");

    printf("\nEnter choice: ");
    scanf("%d",&ch);

    switch(ch)
    {
        case 1:
            showBitVector();
            break;

        case 2:
            createFile();
            break;

        case 3:
            showDirectory();
            break;
    }
}

```

```
        case 4:
            exit(0);

        default:
            printf("\nInvalid choice!");
    }

}while(1);

return 0;
}
```

Q2)

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
```

```
int main()
{
    int blocks,n,i,j,head,total=0;
    int req[100];
    int temp;

    printf("Enter total number of disk blocks: ");
    scanf("%d",&blocks);

    printf("Enter number of disk requests: ");
    scanf("%d",&n);
```

```
printf("Enter disk request string:\n");
```

```
for(i=0;i<n;i++)
```

```
    scanf("%d",&req[i]);
```

```
printf("Enter current head position: ");
```

```
scanf("%d",&head);
```

```
/* Sort requests */
```

```
for(i=0;i<n-1;i++)
```

```
    for(j=i+1;j<n;j++)
```

```
        if(req[i]>req[j])
```

```
        {
```

```
            temp=req[i];
```

```
            req[i]=req[j];
```

```
            req[j]=temp;
```

```
        }
```

```
printf("\nC-SCAN Request Order:\n");
```

```
printf("%d",head);
```

```
/* Service requests towards right */
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    if(req[i]>=head)
```

```
    {
```

```
        total += abs(head-req[i]);
```

```
        head=req[i];
```

```

        printf(" -> %d",head);
    }
}

/* Move to end of disk */
if(head != blocks-1)
{
    total += abs(head-(blocks-1));
    head=blocks-1;
    printf(" -> %d",head);
}
/* Jump to beginning */
total += blocks-1;
head=0;
printf(" -> %d",head);
/* Service remaining requests */
for(i=0;i<n;i++)
{
    if(req[i]<100)
    {
        total += abs(head-req[i]);
        head=req[i];
        printf(" -> %d",head);
    }
}
printf("\n\nTotal Head Movements = %d\n",total);

return 0;
}

```

Slip 7:-

Q.1 Consider the following snapshot of the system.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	3	3	2	1
P1	3	1	2	1	5	2	5	2				
P2	2	1	0	3	2	3	1	6				
P3	1	3	1	2	1	4	2	4				
P4	1	4	3	2	3	6	6	5				

Using Resource–Request algorithm to Check whether the current system is in safe state or not

[15 Marks]

Q2. Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

[15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
int alloc[5][4] = {
```

```
    {2,0,0,1},
```

```
    {3,1,2,1},
```

```
    {2,1,0,3},
```

```
    {1,3,1,2},
```

```
    {1,4,3,2}
```

```
};
```

```
int max[5][4] = {
```

```

    {4,2,1,2},
    {5,2,5,2},
    {2,3,1,6},
    {1,4,2,4},
    {3,6,6,5}
};

int avail[4] = {3,3,2,1};
int need[5][4];

void calculateNeed()
{
    int i,j;

    for(i=0;i<5;i++)
        for(j=0;j<4;j++)
            need[i][j]=max[i][j]-alloc[i][j];
}

void displayNeed()
{
    int i,j;

    printf("\nNeed Matrix\n");
    printf("Process\tA\tB\tC\tD\n");

    for(i=0;i<5;i++)
    {
        printf("P%d\t",i);

```

```
        for(j=0;j<4;j++)
            printf("%d\t",need[i][j]);

        printf("\n");
    }
}
```

```
void safety()
{
    int work[4];
    int finish[5]={0};
    int safe[5];
    int i,j,count=0,found;

    for(j=0;j<4;j++)
        work[j]=avail[j];

    while(count<5)
    {
        found=0;

        for(i=0;i<5;i++)
        {
            if(finish[i]==0)
            {
                for(j=0;j<4;j++)
                {
                    if(need[i][j]>work[j])
```

```

        break;
    }

    if(j==4)
    {
        for(j=0;j<4;j++)
            work[j]=work[j]+alloc[i][j];

        finish[i]=1;
        safe[count]=i;
        count++;
        found=1;
    }
}

if(found==0)
    break;
}

if(count==5)
{
    printf("\nSystem is in SAFE state.\n");

    printf("Safe Sequence: ");

    for(i=0;i<5;i++)
        printf("P%d ",safe[i]);

```

```

        printf("\n");
    }
    else
    {
        printf("\nSystem is NOT in SAFE state.\n");
    }
}

```

```

int main()
{
    printf("Available Resources:\n");
    printf("A B C D\n");
    printf("%d %d %d %d\n",
        avail[0],avail[1],avail[2],avail[3]);

    calculateNeed();

    displayNeed();

    safety();

    return 0;
}

```

Q2)

```

#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>

```

```
int main()
{
    pid_t pid;
    int priority;

    printf("Parent Process ID: %d\n",getpid());

    pid=fork();

    if(pid<0)
    {
        printf("Fork Failed\n");
        return 1;
    }

    if(pid==0)
    {
        /* Child process */
        priority=nice(-10);

        printf("\nChild Process\n");
        printf("Child PID: %d\n",getpid());
        printf("Child priority: %d\n",priority);
    }
    else
    {
        /* Parent process */
        priority=nice(10);
```

```
    printf("\nParent Process\n");  
    printf("Parent PID: %d\n",getpid());  
    printf("Parent priority: %d\n",priority);  
}  
  
return 0;  
}
```

Slip 7:-

Q.1 Consider the following snapshot of the system.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	3	3	2	1
P1	3	1	2	1	5	2	5	2				
P2	2	1	0	3	2	3	1	6				
P3	1	3	1	2	1	4	2	4				
P4	1	4	3	2	3	6	6	5				

Using Resource –Request algorithm to Check whether the current system is in safe state or not [15 Marks]

Q2. Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call. [15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
int alloc[5][4] = {
```

```
    {2,0,0,1},
```

```
    {3,1,2,1},
```

```
    {2,1,0,3},
```

```
    {1,3,1,2},
```

```
    {1,4,3,2}
```

```
};
```

```
int max[5][4] = {
```

```
    {4,2,1,2},
```

```
    {5,2,5,2},
```

```
    {2,3,1,6},
```

```
    {1,4,2,4},
```

```

    {3,6,6,5}
};

int avail[4] = {3,3,2,1};
int need[5][4];

void calculateNeed()
{
    int i,j;

    for(i=0;i<5;i++)
        for(j=0;j<4;j++)
            need[i][j]=max[i][j]-alloc[i][j];
}

void displayNeed()
{
    int i,j;

    printf("\nNeed Matrix\n");
    printf("Process\tA\tB\tC\tD\n");

    for(i=0;i<5;i++)
    {
        printf("P%d\t",i);

        for(j=0;j<4;j++)
            printf("%d\t",need[i][j]);
    }
}

```

```
        printf("\n");  
    }  
}
```

```
void safety()  
{  
    int work[4];  
    int finish[5]={0};  
    int safe[5];  
    int i,j,count=0,found;  
  
    for(j=0;j<4;j++)  
        work[j]=avail[j];  
  
    while(count<5)  
    {  
        found=0;  
  
        for(i=0;i<5;i++)  
        {  
            if(finish[i]==0)  
            {  
                for(j=0;j<4;j++)  
                {  
                    if(need[i][j]>work[j])  
                        break;  
                }  
  
                if(j==4)
```

```

        {
            for(j=0;j<4;j++)
                work[j]=work[j]+alloc[i][j];

            finish[i]=1;
            safe[count]=i;
            count++;
            found=1;
        }
    }
}

if(found==0)
    break;
}

if(count==5)
{
    printf("\nSystem is in SAFE state.\n");

    printf("Safe Sequence: ");

    for(i=0;i<5;i++)
        printf("P%d ",safe[i]);

    printf("\n");
}
else
{

```

```
        printf("\nSystem is NOT in SAFE state.\n");
    }
}
```

```
int main()
{
    printf("Available Resources:\n");
    printf("A B C D\n");
    printf("%d %d %d %d\n",
        avail[0],avail[1],avail[2],avail[3]);

    calculateNeed();

    displayNeed();

    safety();

    return 0;
}
```

Q2)

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
```

```
int main()
{
    pid_t pid;
    int priority;
```

```
printf("Parent Process ID: %d\n",getpid());

pid=fork();

if(pid<0)
{
    printf("Fork Failed\n");
    return 1;
}
if(pid==0)
{
    /* Child process */
    priority=nice(-10);
    printf("\nChild Process\n");
    printf("Child PID: %d\n",getpid());
    printf("Child priority: %d\n",priority);
}
else
{
    /* Parent process */
    priority=nice(10);
    printf("\nParent Process\n");
    printf("Parent PID: %d\n",getpid());
    printf("Parent priority: %d\n",priority);
}

return 0;
}
```

Slip 8:-

Q.1 Write a program to simulate Index file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driven program with menu options as mentioned above and implement each option

- Show Bit Vector
- Create New File
- Show Directory
- Exit

[15 Marks]

Q.2 Write a program that demonstrates the use of nice () system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

[15 Marks]

Q1)

```
#include<stdio.h>

#include<stdlib.h>

#include<unistd.h>

#include<string.h>

#include<time.h>
```

```
struct file
{
    char name[20];
    int index;
    int blocks[100];
    int count;
} dir[20];
```

```
int bit[100];
int n,fcount=0;
```

```
void showBitVector()
{
```

```
int i;
```

```
printf("\nBit Vector:\n");
```

```
for(i=0;i<n;i++)
```

```
    printf("%d ",bit[i]);
```

```
printf("\n");
```

```
}
```

```
void createFile()
```

```
{
```

```
    char name[20];
```

```
    int blocks,i,j,count=0,index=-1;
```

```
    printf("\nEnter file name: ");
```

```
    scanf("%s",name);
```

```
    printf("Enter number of blocks: ");
```

```
    scanf("%d",&blocks);
```

```
/* Check file name */
```

```
for(i=0;i<fcount;i++)
```

```
{
```

```
    if(strcmp(dir[i].name,name)==0)
```

```
    {
```

```
        printf("\nFile already exists!\n");
```

```
        return;
```

```
    }
```

```
}
```

```
/*
```

```
One free block is required for the index block  
and 'blocks' free blocks for file data.
```

```
*/
```

```
for(i=0;i<n;i++)
```

```
    if(bit[i]==1)
```

```
        count++;
```

```
if(count<blocks+1)
```

```
{
```

```
    printf("\nNot enough free blocks!\n");
```

```
    return;
```

```
}
```

```
/* Allocate index block */
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    if(bit[i]==1)
```

```
    {
```

```
        index=i;
```

```
        bit[i]=0;
```

```
        break;
```

```
    }
```

```
}
```

```
/* Allocate data blocks */
```

```
count=0;
```

```
for(i=0;i<n && count<blocks;i++)
{
    if(bit[i]==1)
    {
        bit[i]=0;
        dir[fcount].blocks[count]=i;
        count++;
    }
}
```

```
strcpy(dir[fcount].name,name);
dir[fcount].index=index;
dir[fcount].count=blocks;
fcount++;
```

```
printf("\nFile created successfully!");
printf("\nIndex block = %d",index);
printf("\nData blocks = ");
```

```
for(j=0;j<blocks;j++)
    printf("%d ",dir[fcount-1].blocks[j]);
```

```
printf("\n");
}
```

```
void showDirectory()
{
    int i,j;
```

```

printf("\nDirectory:\n");
printf("File\tIndex\tData Blocks\n");

for(i=0;i<fcount;i++)
{
    printf("%s\t%d\t",
        dir[i].name,
        dir[i].index);

    for(j=0;j<dir[i].count;j++)
        printf("%d ",dir[i].blocks[j]);

    printf("\n");
}
}

int main()
{
    int i,ch;

    printf("Enter number of disk blocks: ");
    scanf("%d",&n);

    srand(time(NULL));

    /* 1 = free, 0 = allocated */
    for(i=0;i<n;i++)
        bit[i]=rand()%2;

```

```
do
{
    printf("\n\n--- INDEXED FILE ALLOCATION ---");
    printf("\n1. Show Bit Vector");
    printf("\n2. Create New File");
    printf("\n3. Show Directory");
    printf("\n4. Exit");

    printf("\nEnter choice: ");
    scanf("%d",&ch);

    switch(ch)
    {
        case 1:
            showBitVector();
            break;

        case 2:
            createFile();
            break;

        case 3:
            showDirectory();
            break;

        case 4:
            exit(0);
```

```
        default:
            printf("\nInvalid choice!");
        }

}while(1);

return 0;
}
```

Q2)

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
```

```
int main()
{
    pid_t pid;
    int n;

    printf("Parent Process ID: %d\n",getpid());

    pid=fork();

    if(pid<0)
    {
        printf("Fork Failed\n");
        return 1;
    }
```

```
if(pid==0)
{
    /* Child process */
    n=nice(-10);

    printf("\nChild Process\n");
    printf("Child PID: %d\n",getpid());
    printf("Child Nice Value: %d\n",n);
}
else
{
    /* Parent process */
    printf("\nParent Process\n");
    printf("Parent PID: %d\n",getpid());
}

return 0;
}
```

Slip 9:-

Q.1 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the optimal page replacement algorithm. Assume the memory of 'n' frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

[15 Marks]

Q.2 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. the arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

[15 marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
int main()
```

```
{
```

```
    int page[100],frame[20];
```

```
    int n,m,i,j,k,pos,fault=0;
```

```
    int found,farthest,next;
```

```
    int ref[] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
```

```
    int size = 15;
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d",&m);
```

```
    for(i=0;i<m;i++)
```

```
        frame[i]=-1;
```

```
    printf("\nPage\tFrames\t\tPage Fault\n");
```

```
    printf("-----\n");
```

```
for(i=0;i<size;i++)
{
    page[i]=ref[i];
    found=0;

    /* Check page in frames */
    for(j=0;j<m;j++)
    {
        if(frame[j]==page[i])
        {
            found=1;
            break;
        }
    }

    if(found==0)
    {
        fault++;

        /* Empty frame available */
        pos=-1;

        for(j=0;j<m;j++)
        {
            if(frame[j]==-1)
            {
                pos=j;
                break;
            }
        }
    }
}
```

```
    }  
}
```

```
/* No empty frame - find optimal victim */
```

```
if(pos==-1)
```

```
{
```

```
    farthest=-1;
```

```
    for(j=0;j<m;j++)
```

```
    {
```

```
        next=-1;
```

```
        for(k=i+1;k<size;k++)
```

```
        {
```

```
            if(frame[j]==page[k])
```

```
            {
```

```
                next=k;
```

```
                break;
```

```
            }
```

```
        }
```

```
/* Page not used again */
```

```
if(next==-1)
```

```
{
```

```
    pos=j;
```

```
    break;
```

```
}
```

```
if(next>farthest)
```

```

        {
            farthest=next;
            pos=j;
        }
    }
}

frame[pos]=page[i];
}

printf("%d\t",page[i]);

for(j=0;j<m;j++)
    printf("%d ",frame[j]);

if(found==0)
    printf("\tYES\n");
else
    printf("\tNO\n");
}

printf("-----\n");
printf("Total Page Faults = %d\n",fault);

return 0;
}

```

Q2)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<time.h>
```

```
struct process
```

```
{
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int wt;
```

```
    int tat;
```

```
    int ct;
```

```
    int done;
```

```
};
```

```
int main()
```

```
{
```

```
    struct process p[20];
```

```
    int n,i,j,time=0,count=0;
```

```
    int min,ind;
```

```
    float avgwt=0,avgtat=0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        p[i].pid=i+1;
```

```

printf("\nEnter arrival time for P%d: ",i+1);
scanf("%d",&p[i].at);

printf("Enter first CPU burst for P%d: ",i+1);
scanf("%d",&p[i].bt);

p[i].done=0;
}

srand(time(NULL));

printf("\nNext CPU burst generated randomly:\n");

for(i=0;i<n;i++)
{
    int next=1+rand()%9;

    printf("P%d : %d\n",p[i].pid,next);

    /* Fixed I/O waiting time = 2 */
    p[i].bt=p[i].bt+2+next;
}

printf("\nGantt Chart:\n");

while(count<n)
{
    min=9999;
    ind=-1;

```

```
for(i=0;i<n;i++)
{
    if(p[i].done==0 && p[i].at<=time)
    {
        if(p[i].bt<min)
        {
            min=p[i].bt;
            ind=i;
        }
    }
}
```

```
if(ind== -1)
{
    time++;
    continue;
}
```

```
printf(" | P%d ",p[ind].pid);
```

```
p[ind].wt=time-p[ind].at;
```

```
if(p[ind].wt<0)
    p[ind].wt=0;
```

```
time=time+p[ind].bt;
```

```
p[ind].ct=time;
```

```
p[ind].tat=p[ind].ct-p[ind].at;
```

```
p[ind].done=1;
```

```
count++;
```

```
avgwt=avgwt+p[ind].wt;
```

```
avgtat=avgtat+p[ind].tat;
```

```
}
```

```
printf("|\\n");
```

```
printf("\\nProcess\\tAT\\tBT\\tWT\\tTAT\\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
printf("P%d\\t%d\\t%d\\t%d\\t%d\\n",
```

```
p[i].pid,
```

```
p[i].at,
```

```
p[i].bt,
```

```
p[i].wt,
```

```
p[i].tat);
```

```
}
```

```
printf("\\nAverage Waiting Time = %.2f",avgwt/n);
```

```
printf("\\nAverage Turnaround Time = %.2f\\n",avgtat/n);
```

```
return 0;
```

```
}
```

Slip 10:-

Q.1 Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15Marks]

Q.2 Write a simulation program for disk scheduling using Look algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments. 55, 58, 39, 18, 90, 160, 150, 38.
Start Head Position: 100
Direction: Left [15 Marks]

Q.3. Oral/Viva [5 marks]

Q1)

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<time.h>
```

```
struct process
```

```
{
    int pid;
    int at;
    int bt;
    int rem;
    int wt;
    int tat;
    int ct;
    int done;
};
```

```
int main()
{
    struct process p[20];
    int n,qt,i,time=0,completed=0;
    int queue[1000],front=0,rear=0;
    int visited[20]={0};
    int run;

    float avgwt=0,avgtat=0;

    srand(time(NULL));

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter time quantum: ");
    scanf("%d",&qt);

    for(i=0;i<n;i++)
    {
        p[i].pid=i+1;

        printf("\nEnter arrival time for P%d: ",i+1);
        scanf("%d",&p[i].at);

        printf("Enter first CPU burst for P%d: ",i+1);
        scanf("%d",&p[i].bt);
```

```

/* Generate next CPU burst randomly */
p[i].bt += 1 + rand()%9;

p[i].rem=p[i].bt;
p[i].done=0;
}

printf("\nGenerated CPU Bursts:\n");

for(i=0;i<n;i++)
    printf("P%d = %d\n",p[i].pid,p[i].bt);

/* Add processes which arrive at time 0 */
for(i=0;i<n;i++)
{
    if(p[i].at==0)
    {
        queue[rear++]=i;
        visited[i]=1;
    }
}

printf("\nGantt Chart:\n");

while(completed<n)
{
    /* If ready queue is empty, move time forward */
    if(front==rear)
    {

```

```

time++;

for(i=0;i<n;i++)
{
    if(p[i].visited==0 && p[i].at<=time)
    {
        queue[rear++]=i;
        visited[i]=1;
    }
}

continue;
}

i=queue[front++];

printf(" | P%d ",p[i].pid);

if(p[i].rem>qt)
{
    run=qt;
    p[i].rem-=qt;
}
else
{
    run=p[i].rem;
    p[i].rem=0;
}

```

```
time+=run;
```

```
/* Add newly arrived processes */
```

```
{
```

```
    int j;
```

```
    for(j=0;j<n;j++)
```

```
    {
```

```
        if(visited[j]==0 && p[j].at<=time)
```

```
        {
```

```
            queue[rear++]=j;
```

```
            visited[j]=1;
```

```
        }
```

```
    }
```

```
}
```

```
if(p[i].rem>0)
```

```
{
```

```
    /*
```

```
        * Fixed I/O waiting time = 2 units.
```

```
        * Process returns to ready queue after I/O.
```

```
    */
```

```
    time+=2;
```

```
    queue[rear++]=i;
```

```
}
```

```
else
```

```
{
```

```
    p[i].ct=time;
```

```
    p[i].tat=p[i].ct-p[i].at;
```

```

        p[i].wt=p[i].tat-p[i].bt;

        completed++;
    }
}

printf("|\\n");

printf("\\nProcess\\tAT\\tBT\\tWT\\tTAT\\n");

for(i=0;i<n;i++)
{
    printf("P%d\\t%d\\t%d\\t%d\\t%d\\n",
        p[i].pid,
        p[i].at,
        p[i].bt,
        p[i].wt,
        p[i].tat);

    avgwt+=p[i].wt;
    avgtat+=p[i].tat;
}

printf("\\nAverage Waiting Time = %.2f",avgwt/n);
printf("\\nAverage Turnaround Time = %.2f\\n",avgtat/n);

return 0;
}

```

Q2)

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
```

```
int main()
{
    int blocks,n,i,j,head,total=0;
    int req[100],temp;

    printf("Enter total number of disk blocks: ");
    scanf("%d",&blocks);

    printf("Enter number of disk requests: ");
    scanf("%d",&n);

    printf("Enter disk request string:\n");

    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter current head position: ");
    scanf("%d",&head);

    /* Sort requests */
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(req[i]>req[j])
            {
                temp=req[i];
```

```

        req[i]=req[j];
        req[j]=temp;
    }
    printf("\nLOOK Request Order:\n");
    printf("%d",head);
    /* Move Left */
    for(i=n-1;i>=0;i--)
    {
        if(req[i]<head)
        {
            total+=abs(head-req[i]);
            head=req[i];

            printf(" -> %d",head);
        }
    }
    /* Reverse direction */
    for(i=0;i<n;i++)
    {
        if(req[i]>head)
        {
            total+=abs(head-req[i]);
            head=req[i];
            printf(" -> %d",head);
        }
    }
    printf("\n\nTotal Head Movements = %d\n",total);
    return 0;
}

```

Slip 11:-

- Q.1 Write a C program to simulate Banker's algorithm for the purpose of Deadlock avoidance. The following snapshot of system, A, B, C and D are the resource type.

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

Implement the following Menu.

- Accept Available
 - Display Allocation, Max
 - Display the contents of need matrix
 - Display Available
- [15 Marks]

- Q.2 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the optimal page replacement algorithm. Assume the memory of n frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2 [15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
int alloc[5][3] = {
```

```
    {0,1,0},
```

```
    {2,0,0},
```

```
    {3,0,3},
```

```
    {2,1,1},
```

```
    {0,0,2}
```

```
};
```

```
int max[5][3] = {
```

```

    {0,0,0},
    {2,0,2},
    {0,0,0},
    {1,0,0},
    {0,0,2}
};

int need[5][3];
int avail[3];

void acceptAvailable()
{
    printf("\nEnter Available resources (A B C): ");
    scanf("%d%d%d",&avail[0],&avail[1],&avail[2]);
}

void displayAllocMax()
{
    int i;

    printf("\nProcess\tAllocation\tMax\n");
    printf("\tA B C\t\tA B C\n");

    for(i=0;i<5;i++)
    {
        printf("P%d\t%d %d %d\t\t%d %d %d\n",
            i,
            alloc[i][0],alloc[i][1],alloc[i][2],
            max[i][0],max[i][1],max[i][2]);
    }
}

```

```
}  
}
```

```
void calculateNeed()
```

```
{  
    int i,j;  
  
    for(i=0;i<5;i++)  
        for(j=0;j<3;j++)  
            need[i][j]=max[i][j]-alloc[i][j];  
}
```

```
void displayNeed()
```

```
{  
    int i,j;  
  
    calculateNeed();  
  
    printf("\nNeed Matrix\n");  
    printf("Process\tA\tB\tC\n");  
  
    for(i=0;i<5;i++)  
    {  
        printf("P%d\t",i);  
  
        for(j=0;j<3;j++)  
            printf("%d\t",need[i][j]);  
  
        printf("\n");  
    }
```

```

    }
}

void displayAvailable()
{
    printf("\nAvailable Resources\n");
    printf("A\tB\tC\n");
    printf("%d\t%d\t%d\n",
        avail[0],avail[1],avail[2]);
}

int main()
{
    int ch;

    do
    {
        printf("\n\n--- BANKER'S ALGORITHM ---");
        printf("\n1. Accept Available");
        printf("\n2. Display Allocation, Max");
        printf("\n3. Display Need Matrix");
        printf("\n4. Display Available");
        printf("\n5. Exit");

        printf("\nEnter choice: ");
        scanf("%d",&ch);

        switch(ch)
        {

```

```
case 1:
    acceptAvailable();
    break;

case 2:
    displayAllocMax();
    break;

case 3:
    displayNeed();
    break;

case 4:
    displayAvailable();
    break;

case 5:
    exit(0);

default:
    printf("\nInvalid choice!");
}

}while(1);

return 0;
}
```

Q2)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
int main()
```

```
{
```

```
    int page[15]={8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
```

```
    int frame[20];
```

```
    int n,i,j,k,pos,found;
```

```
    int farthest,next,fault=0;
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
```

```
        frame[i]=-1;
```

```
    printf("\nPage\tFrames\tFault\n");
```

```
    printf("-----\n");
```

```
    for(i=0;i<15;i++)
```

```
    {
```

```
        found=0;
```

```
        /* Check page already present */
```

```
        for(j=0;j<n;j++)
```

```
        {
```

```
            if(frame[j]==page[i])
```

```

    {
        found=1;
        break;
    }
}

if(found==0)
{
    fault++;

    /* Find empty frame */
    pos=-1;

    for(j=0;j<n;j++)
    {
        if(frame[j]==-1)
        {
            pos=j;
            break;
        }
    }

    /* If no empty frame, find optimal page */
    if(pos==-1)
    {
        farthest=-1;

        for(j=0;j<n;j++)
        {

```

```

next=-1;

for(k=i+1;k<15;k++)
{
    if(frame[j]==page[k])
    {
        next=k;
        break;
    }
}

/* Page will not be used again */
if(next== -1)
{
    pos=j;
    break;
}

if(next>farthest)
{
    farthest=next;
    pos=j;
}
}

frame[pos]=page[i];
}

```

```
printf("%d\t",page[i]);

for(j=0;j<n;j++)
    printf("%d ",frame[j]);

if(found)
    printf("\tNo\n");
else
    printf("\tYes\n");
}

printf("-----\n");
printf("Total Page Faults = %d\n",fault);

return 0;
}
```

Slip 12:-

Q.1 Write the program to simulate FCFS CPU scheduling. The arrival time and first CPU- burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15 marks]

Q.2 Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call child process sorts the integers using insertion sort [15 marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<time.h>
```

```
struct process
```

```
{
```

```
    int pid,at,bt,wt,tat,ct;
```

```
};
```

```
int main()
```

```
{
```

```
    struct process p[20];
```

```
    int n,i,j,time=0;
```

```
    int temp;
```

```
    float awt=0,atat=0;
```

```
    srand(time(NULL));
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d",&n);
```

```

for(i=0;i<n;i++)
{
    p[i].pid=i+1;

    printf("\nEnter arrival time for P%d: ",i+1);
    scanf("%d",&p[i].at);

    printf("Enter first CPU burst for P%d: ",i+1);
    scanf("%d",&p[i].bt);

    /* Generate next CPU burst randomly */
    p[i].bt += 1+rand()%9;
}

/* Sort according to arrival time */
for(i=0;i<n-1;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(p[i].at>p[j].at)
        {
            struct process t=p[i];
            p[i]=p[j];
            p[j]=t;
        }
    }
}

```

```
printf("\nGenerated CPU Bursts:\n");
```

```
for(i=0;i<n;i++)
```

```
    printf("P%d = %d\n",p[i].pid,p[i].bt);
```

```
printf("\nGantt Chart:\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    /* CPU remains idle until process arrives */
```

```
    if(time<p[i].at)
```

```
        time=p[i].at;
```

```
    printf(" | P%d ",p[i].pid);
```

```
    p[i].wt=time-p[i].at;
```

```
    time=time+p[i].bt;
```

```
    p[i].ct=time;
```

```
    p[i].tat=p[i].ct-p[i].at;
```

```
    /*
```

```
    * Fixed I/O waiting time = 2 units
```

```
    * before the next CPU burst.
```

```
    */
```

```
    time=time+2;
```

```
}
```

```
printf("|\\n");
```

```
printf("\\nProcess\\tAT\\tBT\\tWT\\tTAT\\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    printf("P%d\\t%d\\t%d\\t%d\\t%d\\n",
```

```
    p[i].pid,p[i].at,p[i].bt,
```

```
    p[i].wt,p[i].tat);
```

```
    awt+=p[i].wt;
```

```
    atat+=p[i].tat;
```

```
}
```

```
printf("\\nAverage Waiting Time = %.2f",awt/n);
```

```
printf("\\nAverage Turnaround Time = %.2f\\n",atat/n);
```

```
return 0;
```

```
}
```

Q2)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
#include<sys/wait.h>
```

```
void bubble(int a[],int n)
```

```
{
```

```
int i,j,temp;
```

```
for(i=0;i<n-1;i++)
```

```
    for(j=0;j<n-i-1;j++)
```

```
        if(a[j]>a[j+1])
```

```
        {
```

```
            temp=a[j];
```

```
            a[j]=a[j+1];
```

```
            a[j+1]=temp;
```

```
        }
```

```
    }
```

```
void insertion(int a[],int n)
```

```
{
```

```
    int i,j,key;
```

```
    for(i=1;i<n;i++)
```

```
    {
```

```
        key=a[i];
```

```
        j=i-1;
```

```
        while(j>=0 && a[j]>key)
```

```
        {
```

```
            a[j+1]=a[j];
```

```
            j--;
```

```
        }
```

```
        a[j+1]=key;
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    int a[50],n,i;
```

```
    pid_t pid;
```

```
    printf("Enter number of integers: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter integers:\n");
```

```
    for(i=0;i<n;i++)
```

```
        scanf("%d",&a[i]);
```

```
    pid=fork();
```

```
    if(pid<0)
```

```
    {
```

```
        printf("Fork Failed\n");
```

```
        return 1;
```

```
    }
```

```
    if(pid==0)
```

```
    {
```

```
        /* Child process */
```

```
        printf("\nChild Process\n");
```

```
        printf("Child PID: %d\n",getpid());
```

```

insertion(a,n);

printf("Sorted using Insertion Sort:\n");

for(i=0;i<n;i++)
    printf("%d ",a[i]);

printf("\n");

exit(0);
}
else
{
    /* Parent process */
    printf("\nParent Process\n");
    printf("Parent PID: %d\n",getpid());

    bubble(a,n);

    printf("Sorted using Bubble Sort:\n");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    printf("\n");
    wait(NULL);
    printf("Parent waited for child process.\n");
}
return 0;
}

```

Slip 13:-

Q.1 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15 Marks]

Q.2 Write a simulation program for disk scheduling using LOOK algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head movements
15, 30, 55, 90, 125, 140, 170
Starting Head Position: 100
Direction: Left

[15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<time.h>
```

```
struct process
```

```
{
```

```
    int pid,at,bt,wt,tat,ct;
```

```
};
```

```
int main()
```

```
{
```

```
    struct process p[20],temp;
```

```
    int n,i,j,time=0;
```

```
    float awt=0,atat=0;
```

```
    srand(time(NULL));
```

```

printf("Enter number of processes: ");
scanf("%d",&n);

for(i=0;i<n;i++)
{
    p[i].pid=i+1;

    printf("\nEnter arrival time for P%d: ",i+1);
    scanf("%d",&p[i].at);

    printf("Enter first CPU burst for P%d: ",i+1);
    scanf("%d",&p[i].bt);

    /* Generate next CPU burst randomly */
    p[i].bt += 1+rand()%9;
}

/* Sort by arrival time */
for(i=0;i<n-1;i++)
    for(j=i+1;j<n;j++)
        if(p[i].at>p[j].at)
        {
            temp=p[i];
            p[i]=p[j];
            p[j]=temp;
        }

printf("\nGenerated CPU Burst:\n");

```

```

for(i=0;i<n;i++)
    printf("P%d = %d\n",p[i].pid,p[i].bt);

/*
    Non-preemptive SJF:
    At each step choose the shortest burst
    among the processes that have arrived.
*/

for(i=0;i<n;i++)
{
    int k=i;

    for(j=i+1;j<n;j++)
    {
        if(p[j].at<=time && p[j].bt<p[k].bt)
            k=j;
    }

    if(p[k].at>time)
        time=p[k].at;

    /* Find shortest job among arrived processes */
    for(j=i;j<n;j++)
    {
        if(p[j].at<=time && p[j].bt<p[k].bt)
            k=j;
    }
}

```

```

temp=p[i];
p[i]=p[k];
p[k]=temp;

printf("\nP%d selected",p[i].pid);

p[i].wt=time-p[i].at;

time+=p[i].bt;

p[i].ct=time;
p[i].tat=p[i].ct-p[i].at;

/* Fixed I/O waiting time = 2 units */
time+=2;
}

printf("\n\nGantt Chart:\n");

for(i=0;i<n;i++)
    printf(" | P%d ",p[i].pid);

printf("\n");

printf("\nProcess\tAT\tBT\tWT\tTAT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\n",

```

```

        p[i].pid,
        p[i].at,
        p[i].bt,
        p[i].wt,
        p[i].tat);

    awt+=p[i].wt;
    atat+=p[i].tat;
}

printf("\nAverage Waiting Time = %.2f",awt/n);
printf("\nAverage Turnaround Time = %.2f\n",atat/n);

return 0;
}

```

Q2)

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int main()
{
    int blocks,n,head,total=0;
    int req[100];
    int i,j,temp;

    printf("Enter total number of disk blocks: ");
    scanf("%d",&blocks);

```

```
printf("Enter number of disk requests: ");
```

```
scanf("%d",&n);
```

```
printf("Enter disk request string:\n");
```

```
for(i=0;i<n;i++)
```

```
    scanf("%d",&req[i]);
```

```
printf("Enter current head position: ");
```

```
scanf("%d",&head);
```

```
/* Sort requests */
```

```
for(i=0;i<n-1;i++)
```

```
{
```

```
    for(j=i+1;j<n;j++)
```

```
    {
```

```
        if(req[i]>req[j])
```

```
        {
```

```
            temp=req[i];
```

```
            req[i]=req[j];
```

```
            req[j]=temp;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\nLOOK Request Order:\n");
```

```
printf("%d",head);
```

```

/* Direction: Left */
for(i=n-1;i>=0;i--)
{
    if(req[i]<head)
    {
        total += abs(head-req[i]);
        head=req[i];

        printf(" -> %d",head);
    }
}

/* Reverse direction */
for(i=0;i<n;i++)
{
    if(req[i]>head)
    {
        total += abs(head-req[i]);
        head=req[i];

        printf(" -> %d",head);
    }
}

printf("\n\nTotal Head Movements = %d\n",total);

return 0;
}

```

Slip 14:-

Q.1. Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array. [15 Marks]

Q.2 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames.
Reference String: 3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6 [15 Marks]

Q1)

Parent.c

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/wait.h>

void sort(int a[],int n)
{
    int i,j,temp;

    for(i=0;i<n-1;i++)
        for(j=0;j<n-i-1;j++)
            if(a[j]>a[j+1])
            {
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
            }
}
```

```

        }
    }

int main()
{
    int a[20],n,i,item;
    pid_t pid;
    char *args[25];

    printf("Enter number of integers: ");
    scanf("%d",&n);

    printf("Enter integers:\n");

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    printf("Enter element to search: ");
    scanf("%d",&item);

    pid=fork();

    if(pid<0)
    {
        printf("Fork Failed\n");
        return 1;
    }

    if(pid==0)

```

```

{
    /* Child process */
    sort(a,n);

    printf("\nChild: Sorted array = ");

    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    printf("\n");

    /* Prepare arguments for execve */
    args[0]="./search";

    for(i=0;i<n;i++)
    {
        args[i+1]=malloc(10);
        sprintf(args[i+1],"%d",a[i]);
    }

    args[n+1]=malloc(10);
    sprintf(args[n+1],"%d",item);

    args[n+2]=NULL;

    execve("./search",args,NULL);

    printf("execve failed\n");
    exit(1);
}

```

```
}  
else  
{  
    wait(NULL);  
    printf("\nParent process completed.\n");  
}  
  
return 0;  
}
```

search.c

```
#include<stdio.h>  
#include<stdlib.h>  
#include<unistd.h>  
  
int main(int argc,char *argv[])  
{  
    int a[20],n,item;  
    int low,high,mid,found=0;  
    int i;  
  
    /*  
        Last argument is the element to search.  
        All previous arguments are sorted array elements.  
    */  
  
    n=argc-2;  
    item=atoi(argv[argc-1]);
```

```
for(i=0;i<n;i++)
    a[i]=atoi(argv[i+1]);

low=0;
high=n-1;

while(low<=high)
{
    mid=(low+high)/2;

    if(a[mid]==item)
    {
        found=1;
        break;
    }
    else if(item<a[mid])
        high=mid-1;
    else
        low=mid+1;
}

printf("\nChild: Binary Search\n");
printf("Element to search = %d\n",item);

if(found)
    printf("Element found at position %d\n",mid+1);
else
    printf("Element not found\n");
```

```
    return 0;
}
```

```
gcc search.c -o search
gcc parent.c -o parent
./parent
```

Q2)

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
```

```
int main()
{
    int page[] = {3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
    int frame[20];
    int n,i,j,pos=0,fault=0,found;

    printf("Enter number of frames: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
        frame[i]=-1;

    printf("\nPage\tFrames\t\tFault\n");
    printf("-----\n");

    for(i=0;i<15;i++)
```

```

{
    found=0;

    /* Check whether page is already present */
    for(j=0;j<n;j++)
    {
        if(frame[j]==page[i])
        {
            found=1;
            break;
        }
    }

    /* Page fault */
    if(found==0)
    {
        frame[pos]=page[i];
        pos=(pos+1)%n;
        fault++;
    }

    printf("%d\t",page[i]);

    for(j=0;j<n;j++)
        printf("%d ",frame[j]);

    if(found==0)
        printf("\tYes\n");
    else

```

```
        printf("\tNo\n");
    }

    printf("-----\n");
    printf("Total Page Faults = %d\n",fault);

    return 0;
}
```

Slip 15:-

Q.1 Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid) [15 Marks]

Q.2 Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time . [15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    printf("Parent Process Started\n");
```

```
    printf("Parent PID : %d\n",getpid());
```

```
    pid=fork();
```

```
    if(pid<0)
```

```
    {
```

```
        printf("Fork Failed\n");
```

```
        return 1;
```

```
    }
```

```
if(pid==0)
{
    /* Child Process */
    printf("\nChild Process Started\n");
    printf("Child PID : %d\n",getpid());
    printf("Parent PID : %d\n",getppid());

    sleep(5);

    printf("\nAfter Parent Terminates:\n");
    printf("Child PID : %d\n",getpid());
    printf("New Parent PID : %d\n",getppid());
    printf("Child has become an Orphan Process.\n");
}
else
{
    /* Parent Process */
    printf("\nParent Process\n");
    printf("Parent PID : %d\n",getpid());

    printf("Parent is terminating...\n");
    exit(0);
}

return 0;
}
```

Q2)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<time.h>
```

```
struct process
```

```
{
```

```
    int pid,at,prio;
```

```
    int bt1,bt2,rem;
```

```
    int firstdone,io,done;
```

```
    int ct,wt,tat;
```

```
};
```

```
int main()
```

```
{
```

```
    struct process p[20];
```

```
    int n,i,time=0,completed=0;
```

```
    int current=-1,best;
```

```
    float awt=0,atat=0;
```

```
    srand(time(NULL));
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        p[i].pid=i+1;
```

```

printf("\nEnter Arrival Time, First CPU Burst and Priority for P%d: ",
    i+1);
scanf("%d%d%d",&p[i].at,&p[i].bt1,&p[i].prio);

p[i].bt2=1+rand()%9;
p[i].rem=p[i].bt1;

p[i].firstdone=0;
p[i].io=0;
p[i].done=0;
}

printf("\nGenerated Next CPU Bursts:\n");

for(i=0;i<n;i++)
    printf("P%d = %d\n",p[i].pid,p[i].bt2);

printf("\nGantt Chart:\n");

while(completed<n)
{
    /* I/O waiting */
    for(i=0;i<n;i++)
    {
        if(p[i].io>0)
        {
            p[i].io--;

```

```

        if(p[i].io==0)
            p[i].rem=p[i].bt2;
    }
}

/* Find highest priority ready process */
best=-1;

for(i=0;i<n;i++)
{
    if(!p[i].done &&
        p[i].at<=time &&
        p[i].io==0 &&
        p[i].rem>0)
    {
        if(best==-1 || p[i].prio<p[best].prio)
            best=i;
    }
}

if(best!=-1)
{
    time++;
    current=-1;
    continue;
}

/* New process / preemption */
if(current!=best)

```

```

{
    printf(" | P%d ",p[best].pid);
    current=best;
}

/* Run for one unit */
p[best].rem--;
time++;

/* First CPU burst completed */
if(p[best].rem==0 && p[best].firstdone==0)
{
    p[best].firstdone=1;
    p[best].io=2;
    current=-1;
}

/* Second CPU burst completed */
else if(p[best].rem==0 && p[best].firstdone==1)
{
    p[best].done=1;
    p[best].ct=time;
    completed++;
    current=-1;
}
}

printf("|\\n");

```

```

for(i=0;i<n;i++)
{
    p[i].tat=p[i].ct-p[i].at;

    /* Total CPU time = first burst + random second burst */
    p[i].wt=p[i].tat-p[i].bt1-p[i].bt2-2;
}

printf("\nProcess\tAT\tPriority\tWT\tTAT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t\t%d\t%d\n",
        p[i].pid,
        p[i].at,
        p[i].prio,
        p[i].wt,
        p[i].tat);

    awt+=p[i].wt;
    atat+=p[i].tat;
}

printf("\nAverage Waiting Time = %.2f",awt/n);
printf("\nAverage Turnaround Time = %.2f\n",atat/n);

return 0;
}

```

Slip 16:-

Q.1 Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.

Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

[15 Marks]

Q.2 Write a C program that behaves like a shell which displays the command prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'count' as

- a \$ count c filename: It will display the number of characters in given file
- b \$ count w filename: It will display the number of words in given file
- c \$ count l filename: It will display the number of lines in given file

[15 Marks]

Q1)

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
```

```
struct frame
```

```
{
    int page;
    int counter;
};
```

```
int main()
```

```
{
    int ref[] = {3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
    int size=15;
    struct frame f[20];

    int n,i,j,pos,found;
```

```

int time=0,fault=0;

printf("Enter number of frames: ");
scanf("%d",&n);

/* Initialize frames */
for(i=0;i<n;i++)
{
    f[i].page=-1;
    f[i].counter=0;
}

printf("\nPage\tFrames\t\tCounter\t\tFault\n");
printf("-----\n");

for(i=0;i<size;i++)
{
    time++;
    found=0;

    /* Check whether page is already present */
    for(j=0;j<n;j++)
    {
        if(f[j].page==ref[i])
        {
            found=1;

            /* Update counter on reference */
            f[j].counter=time;

```

```

        break;
    }
}

if(found==0)
{
    fault++;

    /* Find free frame */
    pos=-1;

    for(j=0;j<n;j++)
    {
        if(f[j].page== -1)
        {
            pos=j;
            break;
        }
    }

    /* No free frame: find least recently used */
    if(pos== -1)
    {
        pos=0;

        for(j=1;j<n;j++)
        {
            if(f[j].counter<f[pos].counter)
                pos=j;
        }
    }
}

```

```

        }
    }

    f[pos].page=ref[i];
    f[pos].counter=time;
}

printf("%d\t",ref[i]);

for(j=0;j<n;j++)
    printf("%d ",f[j].page);

printf("\t");

for(j=0;j<n;j++)
    printf("%d ",f[j].counter);

if(found==0)
    printf("\tYES\n");
else
    printf("\tNO\n");
}

printf("-----\n");
printf("Total Page Faults = %d\n",fault);

return 0;
}

```

Q2)

```
#include<stdio.h>
```

```
#include<string.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
#include<sys/wait.h>
```

```
int count(char *option,char *file)
```

```
{
```

```
    FILE *fp;
```

```
    int ch,characters=0,words=0,lines=0,inword=0;
```

```
    fp=fopen(file,"r");
```

```
    if(fp==NULL)
```

```
    {
```

```
        printf("Unable to open file\n");
```

```
        return 1;
```

```
    }
```

```
    while((ch=fgetc(fp))!=EOF)
```

```
    {
```

```
        characters++;
```

```
        if(ch=='\n')
```

```
            lines++;
```

```
        if(ch==' ' || ch=='\t' || ch=='\n')
```

```
        inword=0;
    else if(inword==0)
    {
        words++;
        inword=1;
    }
}
```

```
fclose(fp);
```

```
if(strcmp(option,"c")==0)
    printf("Characters = %d\n",characters);
else if(strcmp(option,"w")==0)
    printf("Words = %d\n",words);
else if(strcmp(option,"l")==0)
    printf("Lines = %d\n",lines);
else
    printf("Invalid count option\n");

return 0;
}
```

```
int main()
{
    char command[100];
    char *args[20];
    char *token;
    int n,i,status;
    pid_t pid;
```

```

while(1)
{
    printf("$ ");
    fflush(stdout);

    fgets(command,100,stdin);

    command[strcspn(command,"\n")]='\0';

    if(strlen(command)==0)
        continue;

    n=0;
    token= strtok(command," ");

    while(token!=NULL)
    {
        args[n++]=token;
        token= strtok(NULL," ");
    }

    args[n]=NULL;

    /* Exit shell */
    if(strcmp(args[0],"exit")==0)
        break;

    /* count command */
    if(strcmp(args[0],"count")==0)

```

```

{
    if(n!=3)
    {
        printf("Usage: count c/w/l filename\n");
        continue;
    }
    count(args[1],args[2]);
    continue;
}
/* Execute other commands */
pid=fork();

if(pid<0)
{
    printf("Fork failed\n");
}
else if(pid==0)
{
    execvp(args[0],args);
    printf("Command not found\n");
    exit(1);
}
else
{
    waitpid(pid,&status,0);
}
}
return 0;
}

```

Slip 17:-

Q.1 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time [15 Marks]

Q.2 Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks. Write menu driven program with menu options as mentioned below and implement each option.

- a. Show Bit Vector
- b. Create New File
- c. Show Directory
- d. Delete File
- e. Exit

[15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<time.h>
```

```
struct process
```

```
{
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int original_bt;
```

```
    int next_bt;
```

```
    int wt;
```

```
    int tat;
```

```
    int ct;
```

```
    int done;
```

```
};
```

```

int main()
{
    struct process p[20],temp;
    int n,i,j,time=0,completed=0;
    int best;
    float awt=0,atat=0;

    srand(time(NULL));

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        p[i].pid=i+1;

        printf("\nEnter arrival time for P%d: ",i+1);
        scanf("%d",&p[i].at);

        printf("Enter first CPU burst for P%d: ",i+1);
        scanf("%d",&p[i].bt);

        p[i].original_bt=p[i].bt;

        /* Generate next CPU burst randomly */
        p[i].next_bt=1+rand()%9;

        p[i].done=0;
    }
}

```

```
}
```

```
printf("\nGenerated Next CPU Bursts:\n");
```

```
for(i=0;i<n;i++)
```

```
    printf("P%d = %d\n",p[i].pid,p[i].next_bt);
```

```
printf("\nGantt Chart:\n");
```

```
while(completed<n)
```

```
{
```

```
    best=-1;
```

```
    /* Find shortest burst among arrived processes */
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        if(!p[i].done && p[i].at<=time)
```

```
        {
```

```
            if(best== -1 ||
```

```
                p[i].bt<p[best].bt)
```

```
                best=i;
```

```
        }
```

```
    }
```

```
    /* No process has arrived */
```

```
    if(best== -1)
```

```
    {
```

```
        time++;
```

```
        continue;
```

```
}
```

```
printf(" | P%d ",p[best].pid);
```

```
/* Waiting time */
```

```
p[best].wt=time-p[best].at;
```

```
/* Execute current CPU burst completely */
```

```
time=time+p[best].bt;
```

```
/* Fixed I/O waiting time = 2 units */
```

```
time=time+2;
```

```
/*
```

```
 * Generate/use the next CPU burst.
```

```
 * Since this is non-preemptive, the current
```

```
 * CPU burst is completed before the next process.
```

```
 */
```

```
p[best].bt=p[best].next_bt;
```

```
/* Execute next CPU burst completely */
```

```
printf(" | P%d ",p[best].pid);
```

```
time=time+p[best].bt;
```

```
p[best].ct=time;
```

```
p[best].tat=p[best].ct-p[best].at;
```

```
p[best].done=1;
```

```
    completed++;

    awt+=p[best].wt;
    atat+=p[best].tat;
}

printf("|\\n");

printf("\\nProcess\\tAT\\tFirstBT\\tNextBT\\tWT\\tTAT\\n");

for(i=0;i<n;i++)
{
    printf("P%d\\t%d\\t%d\\t%d\\t%d\\t%d\\n",
        p[i].pid,
        p[i].at,
        p[i].original_bt,
        p[i].next_bt,
        p[i].wt,
        p[i].tat);
}

printf("\\nAverage Waiting Time = %.2f",awt/n);
printf("\\nAverage Turnaround Time = %.2f\\n",atat/n);

return 0;
}
```

Q2)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<string.h>
```

```
#include<time.h>
```

```
struct file
```

```
{
```

```
    char name[20];
```

```
    int start;
```

```
    int length;
```

```
};
```

```
int main()
```

```
{
```

```
    int n,i,j,choice,len,start,found;
```

```
    int bit[100];
```

```
    char name[20];
```

```
    struct file dir[20];
```

```
    int count=0;
```

```
    srand(time(NULL));
```

```
    printf("Enter number of disk blocks: ");
```

```
    scanf("%d",&n);
```

```
    /* Randomly mark blocks as free(1) or allocated(0) */
```

```
    for(i=0;i<n;i++)
```

```
bit[i]=rand()%2;
```

```
while(1)
```

```
{
```

```
    printf("\n\n--- Sequential File Allocation ---");
```

```
    printf("\n1. Show Bit Vector");
```

```
    printf("\n2. Create New File");
```

```
    printf("\n3. Show Directory");
```

```
    printf("\n4. Delete File");
```

```
    printf("\n5. Exit");
```

```
    printf("\nEnter choice: ");
```

```
    scanf("%d",&choice);
```

```
    switch(choice)
```

```
{
```

```
    case 1:
```

```
        printf("\nBit Vector:\n");
```

```
        for(i=0;i<n;i++)
```

```
            printf("%d ",bit[i]);
```

```
        printf("\n");
```

```
        break;
```

```
    case 2:
```

```
        printf("\nEnter file name: ");
```

```
        scanf("%s",name);
```

```
        printf("Enter number of blocks: ");
```

```
scanf("%d",&len);

start=-1;

/* Search for consecutive free blocks */
for(i=0;i<=n-len;i++)
{
    found=1;

    for(j=0;j<len;j++)
    {
        if(bit[i+j]==0)
        {
            found=0;
            break;
        }
    }

    if(found==1)
    {
        start=i;
        break;
    }
}

if(start==-1)
{
    printf("Contiguous blocks not available.\n");
}
```

```

else
{
    /* Allocate blocks */
    for(i=start;i<start+len;i++)
        bit[i]=0;

    strcpy(dir[count].name,name);
    dir[count].start=start;
    dir[count].length=len;
    count++;

    printf("File created successfully.");
    printf("\nStarting Block = %d",start);
    printf("\nLength = %d blocks\n",len);
}
break;

```

case 3:

```

printf("\nDirectory:\n");
printf("File\tStart\tLength\n");

if(count==0)
    printf("Directory is empty.\n");

for(i=0;i<count;i++)
{
    printf("%s\t%d\t%d\n",
        dir[i].name,
        dir[i].start,

```

```
        dir[i].length);  
    }  
    break;
```

case 4:

```
    printf("\nEnter file name to delete: ");  
    scanf("%s",name);
```

```
    found=-1;
```

```
    for(i=0;i<count;i++)  
    {  
        if(strcmp(dir[i].name,name)==0)  
        {  
            found=i;  
            break;  
        }  
    }  
}
```

```
if(found== -1)  
{  
    printf("File not found.\n");  
}
```

```
else
```

```
{  
    /* Release allocated blocks */  
    for(i=dir[found].start;  
        i<dir[found].start+dir[found].length;  
        i++)
```

```

    {
        bit[i]=1;
    }

    /* Remove directory entry */
    for(i=found;i<count-1;i++)
        dir[i]=dir[i+1];

    count--;

    printf("File deleted successfully.\n");
}

break;

case 5:
    printf("\nExiting...\n");
    return 0;

default:
    printf("\nInvalid choice.\n");
}
}

return 0;
}

```

Slip 18:-

Q.1. Write the program to simulate Preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time [15 Marks]

Q.3 Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid()) [15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<time.h>
```

```
struct process
```

```
{
```

```
    int pid,at,bt1,bt2,rem;
```

```
    int firstdone,io,done;
```

```
    int ct,wt,tat;
```

```
};
```

```
int main()
```

```
{
```

```
    struct process p[20];
```

```
    int n,i,time=0,completed=0;
```

```
    int best,current=-1;
```

```
    float awt=0,atat=0;
```

```
    srand(time(NULL));
```

```

printf("Enter number of processes: ");
scanf("%d",&n);

for(i=0;i<n;i++)
{
    p[i].pid=i+1;

    printf("\nEnter arrival time for P%d: ",i+1);
    scanf("%d",&p[i].at);

    printf("Enter first CPU burst for P%d: ",i+1);
    scanf("%d",&p[i].bt1);

    /* Generate next CPU burst randomly */
    p[i].bt2=1+rand()%9;

    p[i].rem=p[i].bt1;
    p[i].firstdone=0;
    p[i].io=0;
    p[i].done=0;
}

printf("\nGenerated Next CPU Bursts:\n");

for(i=0;i<n;i++)
    printf("P%d = %d\n",p[i].pid,p[i].bt2);

printf("\nGantt Chart:\n");

```

```

while(completed<n)
{
    /*
    * Decrease I/O waiting time.
    * I/O waiting time is fixed at 2 units.
    */
    for(i=0;i<n;i++)
    {
        if(p[i].io>0)
        {
            p[i].io--;

            if(p[i].io==0)
                p[i].rem=p[i].bt2;
        }
    }

    /* Find process with shortest remaining burst */
    best=-1;

    for(i=0;i<n;i++)
    {
        if(!p[i].done &&
            p[i].at<=time &&
            p[i].io==0 &&
            p[i].rem>0)
        {
            if(best==-1 || p[i].rem<p[best].rem)

```

```

        best=i;
    }
}

/* No process is ready */
if(best== -1)
{
    if(current!= -1)
    {
        printf(" | ");
        current=-1;
    }

    time++;
    continue;
}

/* Display process when CPU selection changes */
if(current!=best)
{
    if(current!= -1)
        printf(" | ");

    printf("P%d",p[best].pid);
    current=best;
}

/* Execute for one unit */
p[best].rem--;

```

```

time++;

/* First CPU burst completed */
if(p[best].rem==0 && p[best].firstdone==0)
{
    p[best].firstdone=1;

    /* Fixed I/O waiting time */
    p[best].io=2;

    current=-1;
}

/* Second CPU burst completed */
else if(p[best].rem==0 && p[best].firstdone==1)
{
    p[best].done=1;
    p[best].ct=time;

    completed++;
    current=-1;
}
}

printf(" |\n");

/* Calculate WT and TAT */
for(i=0;i<n;i++)
{

```

```

p[i].tat=p[i].ct-p[i].at;

/*
 * Waiting =
 * Turnaround - total CPU time - I/O waiting
 */
p[i].wt=p[i].tat-p[i].bt1-p[i].bt2-2;

awt+=p[i].wt;
atat+=p[i].tat;
}

printf("\nProcess\tAT\tBT1\tBT2\tWT\tTAT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].at,
        p[i].bt1,
        p[i].bt2,
        p[i].wt,
        p[i].tat);
}

printf("\nAverage Waiting Time = %.2f",awt/n);
printf("\nAverage Turnaround Time = %.2f\n",atat/n);

return 0;

```

```
}
```

Q2)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    printf("Parent Process\n");
```

```
    printf("Parent PID = %d\n",getpid());
```

```
    pid=fork();
```

```
    if(pid<0)
```

```
    {
```

```
        printf("Fork Failed\n");
```

```
        return 1;
```

```
    }
```

```
    if(pid==0)
```

```
    {
```

```
        /* Child process */
```

```
        printf("\nChild Process\n");
```

```
        printf("Child PID = %d\n",getpid());
```

```
        printf("Parent PID = %d\n",getppid());
```

```
sleep(5);

printf("\nAfter Parent Terminates:\n");
printf("Child PID = %d\n",getpid());
printf("New Parent PID = %d\n",getppid());

printf("Child has become an orphan process.\n");
}
else
{
    /* Parent process */
    printf("\nParent is terminating...\n");

    exit(0);
}

return 0;
}
```

Slip 19:-

Q.1 Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated. [15 Marks]

Q.2 Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call [15 Marks]

Q1)

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<time.h>
```

```
struct process
```

```
{
    int pid;
    int at;
    int priority;
    int bt1;
    int bt2;
    int rem;
    int first_done;
    int io;
    int completed;
};
```

```
int main()
```

```

{
    struct process p[20];
    int n,i,time=0,completed=0;
    int best,current=-1;

    srand(time(NULL));

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        p[i].pid=i+1;

        printf("\nEnter Arrival Time for P%d: ",i+1);
        scanf("%d",&p[i].at);

        printf("Enter First CPU Burst for P%d: ",i+1);
        scanf("%d",&p[i].bt1);

        printf("Enter Priority for P%d: ",i+1);
        scanf("%d",&p[i].priority);

        /* Generate next CPU burst randomly */
        p[i].bt2=1+rand()%9;

        p[i].rem=p[i].bt1;
        p[i].first_done=0;
        p[i].io=0;
    }
}

```

```

    p[i].completed=0;
}

printf("\nGenerated Next CPU Bursts:\n");

for(i=0;i<n;i++)
    printf("P%d = %d\n",p[i].pid,p[i].bt2);

printf("\nGantt Chart:\n");

while(completed<n)
{
    /* Handle I/O waiting */
    for(i=0;i<n;i++)
    {
        if(p[i].io>0)
        {
            p[i].io--;

            if(p[i].io==0)
                p[i].rem=p[i].bt2;
        }
    }
}

/* Find highest-priority ready process */
best=-1;

for(i=0;i<n;i++)
{

```

```

if(p[i].completed==0 &&
    p[i].at<=time &&
    p[i].io==0 &&
    p[i].rem>0)
{
    if(best==-1 ||
        p[i].priority<p[best].priority)
    {
        best=i;
    }
}
}

```

/* No process is ready */

```

if(best==-1)
{
    time++;
    current=-1;
    continue;
}

```

/* Display process when process changes */

```

if(current!=best)
{
    printf("| P%d ",p[best].pid);
    current=best;
}

```

/* Execute for one unit */

```

p[best].rem--;
time++;

/* First CPU burst completed */
if(p[best].rem==0 && p[best].first_done==0)
{
    p[best].first_done=1;

    /* I/O waiting time = 2 units */
    p[best].io=2;

    current=-1;
}

/* Next CPU burst completed */
else if(p[best].rem==0 && p[best].first_done==1)
{
    p[best].completed=1;
    completed++;
    current=-1;
}
}

printf("|\\n");

printf("\\nProcess\\tArrival\\tPriority\\tBT1\\tBT2\\n");

for(i=0;i<n;i++)
{

```

```

        printf("P%d\t%d\t%d\t\t%d\t%d\n",
            p[i].pid,
            p[i].at,
            p[i].priority,
            p[i].bt1,
            p[i].bt2);
    }

    return 0;
}

```

Q2)

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>

int main()
{
    pid_t pid;
    int n;

    printf("Parent Process\n");
    printf("Parent PID = %d\n",getpid());

    pid=fork();

    if(pid<0)
    {

```

```
    printf("Fork Failed\n");
    return 1;
}

if(pid==0)
{
    /* Child process */
    n=nice(-10);

    printf("\nChild Process\n");
    printf("Child PID = %d\n",getpid());
    printf("Child Nice Value = %d\n",n);
    printf("Higher priority assigned to child.\n");
}
else
{
    /* Parent process */
    printf("\nParent Process\n");
    printf("Parent PID = %d\n",getpid());
}

return 0;
}
```

Slip 20:-

Q.1 Write a simulation program for disk scheduling using LOOK algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

86, 147, 91, 177, 45, 12, 130

Starting Head Position: 60

Direction: Right

[15 Marks]

Q.2 Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.

Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

[15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
int main()
```

```
{
```

```
    int blocks,n,head,total=0;
```

```
    int req[100];
```

```
    int i,j,temp;
```

```
    printf("Enter total number of disk blocks: ");
```

```
    scanf("%d",&blocks);
```

```
    printf("Enter number of disk requests: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter disk request string:\n");
```

```
    for(i=0;i<n;i++)
```

```
scanf("%d",&req[i]);
```

```
printf("Enter current head position: ");
```

```
scanf("%d",&head);
```

```
/* Sort requests */
```

```
for(i=0;i<n-1;i++)
```

```
{
```

```
    for(j=i+1;j<n;j++)
```

```
    {
```

```
        if(req[i]>req[j])
```

```
        {
```

```
            temp=req[i];
```

```
            req[i]=req[j];
```

```
            req[j]=temp;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\nLOOK Request Order:\n");
```

```
printf("%d",head);
```

```
/* Direction: Right */
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    if(req[i]>head)
```

```
    {
```

```
        total += abs(head-req[i]);
```

```
        head=req[i];
```

```

        printf("-> %d",head);
    }
}

/* Reverse direction */
for(i=n-1;i>=0;i--)
{
    if(req[i]<head)
    {
        total += abs(head-req[i]);
        head=req[i];

        printf("-> %d",head);
    }
}

printf("\n\nTotal Head Movements = %d\n",total);

return 0;
}

```

Q2)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct frame
```

```

{
    int page;

```

```

    int counter;
};

int main()
{
    int ref[] = {3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
    int n=15;
    struct frame f[20];

    int frames,i,j,pos,found;
    int counter=0,fault=0;

    printf("Enter number of frames: ");
    scanf("%d",&frames);

    for(i=0;i<frames;i++)
    {
        f[i].page=-1;
        f[i].counter=0;
    }

    printf("\nPage\tFrames\t\tCounter\t\tFault\n");
    printf("-----\n");

    for(i=0;i<n;i++)
    {
        counter++;
        found=0;

```

```
/* Check page in frames */  
for(j=0;j<frames;j++)  
{  
    if(f[j].page==ref[i])  
    {  
        found=1;  
        f[j].counter=counter;  
        break;  
    }  
}
```

```
/* Page fault */  
if(found==0)  
{  
    fault++;  
    pos=-1;
```

```
/* Find empty frame */  
for(j=0;j<frames;j++)  
{  
    if(f[j].page==-1)  
    {  
        pos=j;  
        break;  
    }  
}
```

```
/* Find LRU page */  
if(pos==-1)
```

```

{
    pos=0;

    for(j=1;j<frames;j++)
    {
        if(f[j].counter<f[pos].counter)
            pos=j;
    }
}

f[pos].page=ref[i];
f[pos].counter=counter;
}

printf("%d\t",ref[i]);

for(j=0;j<frames;j++)
    printf("%d ",f[j].page);

printf("\t");

for(j=0;j<frames;j++)
    printf("%d ",f[j].counter);

if(found==0)
    printf("\tYES\n");
else
    printf("\tNO\n");
}

```

```
printf("-----\n");  
printf("Total Page Faults = %d\n",fault);  
  
return 0;  
}
```

Slip 21:-

Q.1 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2 [15 Marks]

Q.2 Write a program that demonstrates the use of nice() system call.

After a child process is started using fork(), assign higher priority to the child using nice() system call. [15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct frame
```

```
{
```

```
    int page;
```

```
    int freq;
```

```
};
```

```
int main()
```

```
{
```

```
    int ref[] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
```

```
    int n=15;
```

```
    struct frame f[20];
```

```
    int frames,i,j,pos,found;
```

```
    int fault=0;
```

```

printf("Enter number of frames: ");
scanf("%d",&frames);

for(i=0;i<frames;i++)
{
    f[i].page=-1;
    f[i].freq=0;
}

printf("\nPage\tFrames\t\tFrequency\tFault\n");
printf("-----\n");

for(i=0;i<n;i++)
{
    found=0;

    /* Check whether page is already present */
    for(j=0;j<frames;j++)
    {
        if(f[j].page==ref[i])
        {
            found=1;
            f[j].freq++;
            break;
        }
    }

    /* Page fault */
    if(found==0)

```

```

{
    fault++;

    /* Find empty frame */
    pos=-1;

    for(j=0;j<frames;j++)
    {
        if(f[j].page== -1)
        {
            pos=j;
            break;
        }
    }

    /* Find MFU page */
    if(pos== -1)
    {
        pos=0;

        for(j=1;j<frames;j++)
        {
            if(f[j].freq>f[pos].freq)
                pos=j;
        }
    }

    f[pos].page=ref[i];
    f[pos].freq=1;

```

```

    }

    printf("%d\t",ref[i]);

    for(j=0;j<frames;j++)
        printf("%d ",f[j].page);

    printf("\t\t");

    for(j=0;j<frames;j++)
        printf("%d ",f[j].freq);

    if(found==0)
        printf("\tYES\n");
    else
        printf("\tNO\n");
}

printf("-----\n");
printf("Total Page Faults = %d\n",fault);

return 0;
}

```

Q2)

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>

```

```
int main()
{
    pid_t pid;
    int n;

    printf("Parent Process\n");
    printf("Parent PID = %d\n",getpid());

    pid=fork();

    if(pid<0)
    {
        printf("Fork Failed\n");
        return 1;
    }

    if(pid==0)
    {
        /* Child process */
        n=nice(-10);

        printf("\nChild Process\n");
        printf("Child PID = %d\n",getpid());
        printf("Child Nice Value = %d\n",n);
        printf("Higher priority assigned to child.\n");
    }
    else
    {

```

```
/* Parent process */  
printf("\nParent Process\n");  
printf("Parent PID = %d\n",getpid());  
}  
  
return 0;  
}
```

Slip 22:-

Q.1 Write a program to simulate Linked file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driven program with menu options as mentioned below and implement each option.

- a. Show Bit Vector
- b. Create New File
- c. Show Directory
- d. Delete File
- e. Exit

[15 Marks]

Q.2 Write the program to simulate Non-preemptive Priority scheduling.

The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

[15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1)

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<time.h>
```

```
struct file
{
    char name[20];
    int start;
    int end;
    int length;
};
```

```
struct file dir[20];
```

```
int bit[100],next[100];
```

```
int n,count=0;
```

```
void showBitVector()
```

```
{
```

```
    int i;
```

```
    printf("\nBit Vector:\n");
```

```
    for(i=0;i<n;i++)
```

```
        printf("%d ",bit[i]);
```

```
    printf("\n");
```

```
}
```

```
void createFile()
```

```
{
```

```
    char name[20];
```

```
    int blocks[100];
```

```
    int len,i,j,k=0;
```

```
    int start=-1,end=-1;
```

```
    printf("\nEnter file name: ");
```

```
    scanf("%s",name);
```

```
    /* Check duplicate file */
```

```
    for(i=0;i<count;i++)
```

```
{  
    if(strcmp(dir[i].name,name)==0)  
    {  
        printf("File already exists.\n");  
        return;  
    }  
}
```

```
printf("Enter number of blocks: ");  
scanf("%d",&len);
```

```
/* Find required free blocks */
```

```
for(i=0;i<n && k<len;i++)
```

```
{  
    if(bit[i]==1)  
    {  
        blocks[k]=i;  
        k++;  
    }  
}
```

```
if(k<len)
```

```
{  
    printf("Not enough free blocks.\n");  
    return;  
}
```

```
/* Allocate and link blocks */
```

```
for(i=0;i<len;i++)
```

```

{
    bit[blocks[i]]=0;

    if(i<len-1)
        next[blocks[i]]=blocks[i+1];
    else
        next[blocks[i]]=-1;
}

start=blocks[0];
end=blocks[len-1];

strcpy(dir[count].name,name);
dir[count].start=start;
dir[count].end=end;
dir[count].length=len;
count++;

printf("\nFile created successfully.\n");

printf("Linked Blocks: ");

for(i=0;i<len;i++)
    printf("%d ",blocks[i]);

printf("\n");
}

void showDirectory()

```

```

{
    int i,j,current;

    printf("\nDirectory:\n");
    printf("File\tStart\tEnd\tLength\tBlocks\n");

    if(count==0)
    {
        printf("Directory is empty.\n");
        return;
    }

    for(i=0;i<count;i++)
    {
        printf("%s\t%d\t%d\t%d\t",
            dir[i].name,
            dir[i].start,
            dir[i].end,
            dir[i].length);

        current=dir[i].start;

        while(current!=-1)
        {
            printf("%d ",current);
            current=next[current];
        }

        printf("\n");
    }
}

```

```
}  
}
```

```
void deleteFile()
```

```
{
```

```
    char name[20];
```

```
    int i,current,nextBlock;
```

```
    printf("\nEnter file name to delete: ");
```

```
    scanf("%s",name);
```

```
    for(i=0;i<count;i++)
```

```
    {
```

```
        if(strcmp(dir[i].name,name)==0)
```

```
        {
```

```
            current=dir[i].start;
```

```
            /* Release all linked blocks */
```

```
            while(current!=-1)
```

```
            {
```

```
                nextBlock=next[current];
```

```
                bit[current]=1;
```

```
                next[current]=-1;
```

```
                current=nextBlock;
```

```
            }
```

```
            /* Remove directory entry */
```

```

        for(;i<count-1;i++)
            dir[i]=dir[i+1];

        count--;

        printf("File deleted successfully.\n");
        return;
    }
}

printf("File not found.\n");
}

int main()
{
    int i,choice;

    srand(time(NULL));

    printf("Enter number of disk blocks: ");
    scanf("%d",&n);

    /* Randomly mark blocks */
    for(i=0;i<n;i++)
    {
        bit[i]=rand()%2;
        next[i]=-1;
    }
}

```

```
do
{
    printf("\n\n--- LINKED FILE ALLOCATION ---");
    printf("\n1. Show Bit Vector");
    printf("\n2. Create New File");
    printf("\n3. Show Directory");
    printf("\n4. Delete File");
    printf("\n5. Exit");

    printf("\nEnter choice: ");
    scanf("%d",&choice);

    switch(choice)
    {
        case 1:
            showBitVector();
            break;

        case 2:
            createFile();
            break;

        case 3:
            showDirectory();
            break;

        case 4:
            deleteFile();
            break;
```

```

        case 5:
            printf("\nExiting...\n");
            exit(0);

        default:
            printf("\nInvalid choice.\n");
    }

}while(1);

return 0;
}

```

Q2)

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<time.h>

```

```

struct process
{
    int pid,at,priority;
    int bt1,bt2;
    int wt,tat,ct;
    int done;
};

```

```

int main()

```

```

{
    struct process p[20];
    int n,i,time=0,completed=0;
    int best;
    float awt=0,atat=0;

    srand(time(NULL));

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        p[i].pid=i+1;

        printf("\nEnter Arrival Time for P%d: ",i+1);
        scanf("%d",&p[i].at);

        printf("Enter First CPU Burst for P%d: ",i+1);
        scanf("%d",&p[i].bt1);

        printf("Enter Priority for P%d: ",i+1);
        scanf("%d",&p[i].priority);

        /* Generate next CPU burst randomly */
        p[i].bt2=1+rand()%9;

        p[i].done=0;
    }
}

```

```
printf("\nGenerated Next CPU Bursts:\n");
```

```
for(i=0;i<n;i++)
```

```
    printf("P%d = %d\n",p[i].pid,p[i].bt2);
```

```
printf("\nGantt Chart:\n");
```

```
while(completed<n)
```

```
{
```

```
    best=-1;
```

```
    /* Find highest-priority process among arrived */
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        if(!p[i].done && p[i].at<=time)
```

```
        {
```

```
            if(best==-1 ||
```

```
                p[i].priority<p[best].priority)
```

```
            {
```

```
                best=i;
```

```
            }
```

```
        }
```

```
    }
```

```
    /* No process has arrived */
```

```
    if(best==-1)
```

```
    {
```

```
        time++;
```

```

        continue;
    }

    printf(" | P%d ",p[best].pid);

    /* Waiting time */
    p[best].wt=time-p[best].at;

    /* First CPU burst */
    time+=p[best].bt1;

    /*
    * Fixed I/O waiting time = 2 units
    */
    time+=2;

    /* Next CPU burst */
    time+=p[best].bt2;

    p[best].ct=time;
    p[best].tat=p[best].ct-p[best].at;

    p[best].done=1;
    completed++;

    awt+=p[best].wt;
    atat+=p[best].tat;
}

```

```
printf("|\\n");
```

```
printf("\\nProcess\\tAT\\tPriority\\tBT1\\tBT2\\tWT\\tTAT\\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    printf("P%d\\t%d\\t%d\\t\\t%d\\t%d\\t%d\\t%d\\n",
```

```
        p[i].pid,
```

```
        p[i].at,
```

```
        p[i].priority,
```

```
        p[i].bt1,
```

```
        p[i].bt2,
```

```
        p[i].wt,
```

```
        p[i].tat);
```

```
}
```

```
printf("\\nAverage Waiting Time = %.2f",awt/n);
```

```
printf("\\nAverage Turnaround Time = %.2f\\n",atat/n);
```

```
return 0;
```

```
}
```

Slip 23:-

Q.1 Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call. Child process sorts the integers using insertion sort [15 Marks]

Q.2 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
#include<sys/wait.h>
```

```
void bubble(int a[],int n)
```

```
{
```

```
    int i,j,temp;
```

```
    for(i=0;i<n-1;i++)
```

```
        for(j=0;j<n-i-1;j++)
```

```
            if(a[j]>a[j+1])
```

```
            {
```

```
                temp=a[j];
```

```
                a[j]=a[j+1];
```

```
                a[j+1]=temp;
```

```
    }  
}
```

```
void insertion(int a[],int n)
```

```
{  
    int i,j,key;  
  
    for(i=1;i<n;i++)  
    {  
        key=a[i];  
        j=i-1;  
  
        while(j>=0 && a[j]>key)  
        {  
            a[j+1]=a[j];  
            j--;  
        }  
  
        a[j+1]=key;  
    }  
}
```

```
int main()
```

```
{  
    int a[50],n,i;  
    pid_t pid;  
  
    printf("Enter number of integers: ");  
    scanf("%d",&n);
```

```
printf("Enter integers:\n");
```

```
for(i=0;i<n;i++)
```

```
    scanf("%d",&a[i]);
```

```
pid=fork();
```

```
if(pid<0)
```

```
{
```

```
    printf("Fork Failed\n");
```

```
    return 1;
```

```
}
```

```
if(pid==0)
```

```
{
```

```
    /* Child process */
```

```
    insertion(a,n);
```

```
    printf("\nChild Process");
```

```
    printf("\nChild PID = %d",getpid());
```

```
    printf("\nInsertion Sort: ");
```

```
    for(i=0;i<n;i++)
```

```
        printf("%d ",a[i]);
```

```
    printf("\n");
```

```
}
```

```

else
{
    /* Parent process */
    bubble(a,n);

    printf("\nParent Process");
    printf("\nParent PID = %d",getpid());

    printf("\nBubble Sort: ");

    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    printf("\n");

    wait(NULL);

    printf("Parent waited for child.\n");
}

return 0;
}

```

Q2)

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<time.h>

```

```

struct process
{
    int pid,at,bt1,bt2;
    int wt,tat,ct,done;
};

int main()
{
    struct process p[20];
    int n,i,time=0,completed=0,best;
    float awt=0,atat=0;

    srand(time(NULL));

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        p[i].pid=i+1;

        printf("\nEnter arrival time for P%d: ",i+1);
        scanf("%d",&p[i].at);

        printf("Enter first CPU burst for P%d: ",i+1);
        scanf("%d",&p[i].bt1);

        /* Generate next CPU burst randomly */
        p[i].bt2=1+rand()%9;
    }
}

```

```

    p[i].done=0;
}

printf("\nGenerated Next CPU Bursts:\n");

for(i=0;i<n;i++)
    printf("P%d = %d\n",p[i].pid,p[i].bt2);

printf("\nGantt Chart:\n");

while(completed<n)
{
    best=-1;

    /* Find shortest burst among arrived processes */
    for(i=0;i<n;i++)
    {
        if(!p[i].done && p[i].at<=time)
        {
            if(best==-1 || p[i].bt1<p[best].bt1)
                best=i;
        }
    }

    /* No process has arrived */
    if(best==-1)
    {
        time++;
    }
}

```

```

        continue;
    }

    printf(" | P%d ",p[best].pid);

    p[best].wt=time-p[best].at;

    /* First CPU burst */
    time+=p[best].bt1;

    /* I/O waiting time = 2 */
    time+=2;

    /* Next CPU burst */
    time+=p[best].bt2;

    p[best].ct=time;
    p[best].tat=p[best].ct-p[best].at;

    p[best].done=1;
    completed++;

    awt+=p[best].wt;
    atat+=p[best].tat;
}

printf(" |\n");

printf("\nProcess\tAT\tBT1\tBT2\tWT\tTAT\n");

```

```
for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].at,
        p[i].bt1,
        p[i].bt2,
        p[i].wt,
        p[i].tat);
}

printf("\nAverage Waiting Time = %.2f",awt/n);
printf("\nAverage Turnaround Time = %.2f\n",atat/n);

return 0;
}
```

Slip 24:-

Q.1 Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15 Marks]

Q.2 Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array. [15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<time.h>
```

```
struct process
```

```
{
```

```
    int pid,at,bt1,bt2;
```

```
    int rem,wt,tat,ct;
```

```
    int first_done,io,done;
```

```
};
```

```
int main()
```

```
{
```

```
    struct process p[20];
```

```
    int q[1000];
```

```
int n,quantum;
int front=0,rear=0;
int time=0,completed=0;
int i,j,run;
int visited[20]={0};
float awt=0,atat=0;

srand(time(NULL));

printf("Enter number of processes: ");
scanf("%d",&n);

printf("Enter time quantum: ");
scanf("%d",&quantum);

for(i=0;i<n;i++)
{
    p[i].pid=i+1;

    printf("\nEnter arrival time for P%d: ",i+1);
    scanf("%d",&p[i].at);

    printf("Enter first CPU burst for P%d: ",i+1);
    scanf("%d",&p[i].bt1);

    /* Generate next CPU burst randomly */
    p[i].bt2=1+rand()%9;

    p[i].rem=p[i].bt1;
```

```

    p[i].first_done=0;
    p[i].io=0;
    p[i].done=0;
}

printf("\nGenerated Next CPU Bursts:\n");

for(i=0;i<n;i++)
    printf("P%d = %d\n",p[i].pid,p[i].bt2);

/* Add processes arriving at time 0 */
for(i=0;i<n;i++)
{
    if(p[i].at==0)
    {
        q[rear++]=i;
        visited[i]=1;
    }
}

printf("\nGantt Chart:\n");

while(completed<n)
{
    /* Check processes whose I/O has finished */
    for(i=0;i<n;i++)
    {
        if(p[i].io>0)
        {

```

```

        p[i].io--;

        if(p[i].io==0)
            p[i].rem=p[i].bt2;
    }
}

/* Add newly arrived processes */
for(i=0;i<n;i++)
{
    if(!visited[i] &&
        p[i].at<=time)
    {
        q[rear++]=i;
        visited[i]=1;
    }
}

/* CPU idle */
if(front==rear)
{
    time++;
    continue;
}

i=q[front++];

if(p[i].done || p[i].io>0)
    continue;

```

```
printf(" | P%d ",p[i].pid);
```

```
/* Execute for time quantum */
```

```
run=p[i].rem<quantum ? p[i].rem : quantum;
```

```
p[i].rem-=run;
```

```
time+=run;
```

```
/* Add newly arrived processes */
```

```
for(j=0;j<n;j++)
```

```
{
```

```
    if(!visited[j] &&
```

```
        p[j].at<=time)
```

```
    {
```

```
        q[rear++]=j;
```

```
        visited[j]=1;
```

```
    }
```

```
}
```

```
/* CPU burst completed */
```

```
if(p[i].rem==0)
```

```
{
```

```
    if(p[i].first_done==0)
```

```
    {
```

```
        p[i].first_done=1;
```

```
/* I/O waiting = 2 units */
```

```
p[i].io=2;
```

```

    }
    else
    {
        p[i].done=1;
        p[i].ct=time;

        p[i].tat=p[i].ct-p[i].at;
        p[i].wt=p[i].tat-p[i].bt1-p[i].bt2-2;

        completed++;

        awt+=p[i].wt;
        atat+=p[i].tat;
    }
}
else
{
    /* Process not finished, return to ready queue */
    q[rear++]=i;
}
}

printf("|\\n");

printf("\\nProcess\\tAT\\tBT1\\tBT2\\tWT\\tTAT\\n");

for(i=0;i<n;i++)
{
    printf("P%d\\t%d\\t%d\\t%d\\t%d\\t%d\\n",

```

```

        p[i].pid,
        p[i].at,
        p[i].bt1,
        p[i].bt2,
        p[i].wt,
        p[i].tat);
    }

    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f\n",atat/n);

    return 0;
}

```

Q2)

Parent.c

```

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/wait.h>

void sort(int a[],int n)
{
    int i,j,temp;

    for(i=0;i<n-1;i++)
        for(j=0;j<n-i-1;j++)
            if(a[j]>a[j+1])

```

```
        {  
            temp=a[j];  
            a[j]=a[j+1];  
            a[j+1]=temp;  
        }  
    }  
}
```

```
int main()  
{  
    int a[20],n,i,item;  
    char *args[25];  
    pid_t pid;  
  
    printf("Enter number of integers: ");  
    scanf("%d",&n);  
  
    printf("Enter integers:\n");  
  
    for(i=0;i<n;i++)  
        scanf("%d",&a[i]);  
  
    printf("Enter element to search: ");  
    scanf("%d",&item);  
  
    pid=fork();  
  
    if(pid<0)  
    {  
        printf("Fork Failed\n");
```

```

        return 1;
    }

    if(pid==0)
    {
        /* Child process */

        sort(a,n);

        printf("\nSorted Array: ");

        for(i=0;i<n;i++)
            printf("%d ",a[i]);

        printf("\n");

        args[0]="./search";

        for(i=0;i<n;i++)
        {
            args[i+1]=malloc(10);
            sprintf(args[i+1],"%d",a[i]);
        }

        args[n+1]=malloc(10);
        sprintf(args[n+1],"%d",item);

        args[n+2]=NULL;
    }

```

```

    execve("./search",args,NULL);

    printf("execve failed\n");
    exit(1);
}
else
{
    wait(NULL);
    printf("\nParent process completed.\n");
}

return 0;
}

```

Search.c

```

#include<stdio.h>
#include<stdlib.h>

int main(int argc,char *argv[])
{
    int a[20],n,item;
    int low,high,mid,found=0;
    int i;

    /*
        Last argument = element to search
        Previous arguments = sorted array
    */
}

```

```
n=argc-2;
item=atoi(argv[argc-1]);

for(i=0;i<n;i++)
    a[i]=atoi(argv[i+1]);

low=0;
high=n-1;

while(low<=high)
{
    mid=(low+high)/2;

    if(a[mid]==item)
    {
        found=1;
        break;
    }
    else if(item<a[mid])
        high=mid-1;
    else
        low=mid+1;
}

printf("\nBinary Search\n");
printf("Element = %d\n",item);

if(found)
    printf("Element found at position %d\n",mid+1);
```

```
else
```

```
    printf("Element not found\n");
```

```
return 0;
```

```
}
```

Slip 25:-

Q.1 Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid())

[15 Marks]

Q.2 Write a C program that behaves like a shell which displays the command prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'search' as

- a. \$ search f filename pattern : It will search the first occurrence of pattern in the given file
- b. \$ search a filename pattern : It will search all the occurrence of pattern in the given file
- c. \$ search c filename pattern : It will count the number of occurrence of pattern in the given file

[15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    printf("Parent Process\n");
```

```
    printf("Parent PID = %d\n",getpid());
```

```
    pid=fork();
```

```
    if(pid<0)
```

```

{
    printf("Fork Failed\n");
    return 1;
}

if(pid==0)
{
    /* Child process */
    printf("\nChild Process\n");
    printf("Child PID = %d\n",getpid());
    printf("Parent PID = %d\n",getppid());

    sleep(5);

    printf("\nAfter Parent Terminates:\n");
    printf("Child PID = %d\n",getpid());
    printf("New Parent PID = %d\n",getppid());
    printf("Child has become an Orphan Process.\n");
}
else
{
    /* Parent process */
    printf("\nParent is terminating...\n");
    exit(0);
}

return 0;
}

```

Q2)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<string.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
#include<sys/wait.h>
```

```
void search(char option,char *file,char *pattern)
```

```
{
```

```
    FILE *fp;
```

```
    char str[1000];
```

```
    char *p;
```

```
    int count=0;
```

```
    fp=fopen(file,"r");
```

```
    if(fp==NULL)
```

```
    {
```

```
        printf("File not found\n");
```

```
        return;
```

```
    }
```

```
    while(fgets(str,sizeof(str),fp))
```

```
    {
```

```
        p=str;
```

```
        while((p=strstr(p,pattern))!=NULL)
```

```
        {
```

```
count++;

if(option=='f')
{
    printf("First occurrence found\n");
    fclose(fp);
    return;
}

if(option=='a')
    printf("Occurrence %d found\n",count);

/* Move one character to allow overlapping matches */
p++;
}
}

fclose(fp);

if(option=='a')
    printf("Total occurrences = %d\n",count);

if(option=='c')
    printf("Number of occurrences = %d\n",count);

if(option!='f' && option!='a' && option!='c')
    printf("Invalid option\n");
}
```

```
int main()
{
    char command[200];
    char *args[20];
    char *token;
    int n,i;
    pid_t pid;

    while(1)
    {
        printf("$ ");
        fflush(stdout);

        fgets(command,sizeof(command),stdin);

        command[strcspn(command,"\n")]='\0';

        if(strlen(command)==0)
            continue;

        n=0;
        token=strtok(command," ");

        while(token!=NULL)
        {
            args[n++]=token;
            token=strtok(NULL," ");
        }
    }
}
```

```
args[n]=NULL;
```

```
/* Exit shell */
```

```
if(strcmp(args[0],"exit")==0)
```

```
    break;
```

```
/* search command */
```

```
if(strcmp(args[0],"search")==0)
```

```
{
```

```
    if(n!=4)
```

```
    {
```

```
        printf("Usage: search f/a/c filename pattern\n");
```

```
        continue;
```

```
    }
```

```
    search(args[1][0],args[2],args[3]);
```

```
    continue;
```

```
}
```

```
/* Execute normal Linux command */
```

```
pid=fork();
```

```
if(pid<0)
```

```
{
```

```
    printf("Fork Failed\n");
```

```
}
```

```
else if(pid==0)
```

```
{
```

```
    execvp(args[0],args);
```

```
    printf("Command not found\n");  
    exit(1);  
}  
else  
{  
    wait(NULL);  
}  
}  
  
return 0;  
}
```

Slip 26:-

Q.1 Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated [15 Marks]

Q.2 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames.
Reference String: 3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6 [15 Marks]

Q.3. Oral/Viva [05 Marks]

Q1)

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<time.h>
```

```
struct process
{
    int pid;
    int at;
    int priority;
    int bt1, bt2;
    int rem;
    int first_done;
    int io;
    int done;
};
```

```

int main()
{
    struct process p[20];
    int n,i,time=0,completed=0;
    int best,current=-1;

    srand(time(NULL));

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        p[i].pid=i+1;

        printf("\nEnter Arrival Time for P%d: ",i+1);
        scanf("%d",&p[i].at);

        printf("Enter First CPU Burst for P%d: ",i+1);
        scanf("%d",&p[i].bt1);

        printf("Enter Priority for P%d: ",i+1);
        scanf("%d",&p[i].priority);

        /* Generate next CPU burst randomly */
        p[i].bt2=1+rand()%9;

        p[i].rem=p[i].bt1;
        p[i].first_done=0;
    }
}

```

```

    p[i].io=0;
    p[i].done=0;
}

printf("\nGenerated Next CPU Bursts:\n");

for(i=0;i<n;i++)
    printf("P%d = %d\n",p[i].pid,p[i].bt2);

printf("\nGantt Chart:\n");

while(completed<n)
{
    /* Handle I/O waiting */
    for(i=0;i<n;i++)
    {
        if(p[i].io>0)
        {
            p[i].io--;

            if(p[i].io==0)
                p[i].rem=p[i].bt2;
        }
    }
}

/* Find highest-priority ready process */
best=-1;

for(i=0;i<n;i++)

```

```

{
    if(!p[i].done &&
        p[i].at<=time &&
        p[i].io==0 &&
        p[i].rem>0)
    {
        if(best==-1 ||
            p[i].priority<p[best].priority)
        {
            best=i;
        }
    }
}

```

/* No process is ready */

```

if(best==-1)
{
    time++;
    current=-1;
    continue;
}

```

/* Process changed = preemption or new selection */

```

if(current!=best)
{
    printf("| P%d ",p[best].pid);
    current=best;
}

```

```

/* Execute for one time unit */
p[best].rem--;
time++;

/* First CPU burst completed */
if(p[best].rem==0 && p[best].first_done==0)
{
    p[best].first_done=1;

    /* 2 units of I/O waiting */
    p[best].io=2;

    current=-1;
}

/* Next CPU burst completed */
else if(p[best].rem==0 && p[best].first_done==1)
{
    p[best].done=1;
    completed++;
    current=-1;
}
}

printf("|\\n");

printf("\\nProcess\\tArrival\\tPriority\\tBT1\\tBT2\\n");

for(i=0;i<n;i++)

```

```

{
    printf("P%d\t%d\t%d\t\t%d\t%d\n",
        p[i].pid,
        p[i].at,
        p[i].priority,
        p[i].bt1,
        p[i].bt2);
}

return 0;
}

```

Q2)

```
#include<stdio.h>
```

```

int main()
{
    int page[] = {3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
    int n = 15;
    int frame[20];
    int frames;
    int i,j,pos=0,found;
    int faults=0;

    printf("Enter number of frames: ");
    scanf("%d",&frames);

    for(i=0;i<frames;i++)
        frame[i]=-1;

```

```

printf("\nPage\tFrames\t\tFault\n");
printf("-----\n");

for(i=0;i<n;i++)
{
    found=0;

    /* Check whether page is already present */
    for(j=0;j<frames;j++)
    {
        if(frame[j]==page[i])
        {
            found=1;
            break;
        }
    }

    /* Page fault */
    if(found==0)
    {
        frame[pos]=page[i];

        pos=(pos+1)%frames;
        faults++;
    }

    printf("%d\t",page[i]);

```

```
    for(j=0;j<frames;j++)
        printf("%d ",frame[j]);

    if(found==0)
        printf("\tYes\n");
    else
        printf("\tNo\n");
}

printf("-----\n");
printf("Total Page Faults = %d\n",faults);

return 0;
}
```

Slip 27:-

Q.1 Write a simulation program for disk scheduling using SSTF algorithm.

Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

30, 10, 60, 95, 120, 150, 175

Start Head Position: 50

[15 Marks]

Q.2 Write the simulation program to implement demand paging

and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.

Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

[15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int main()
```

```
{
```

```
    int n,i,j;
```

```
    int req[100],visited[100];
```

```
    int head,total=0;
```

```
    int min,diff,pos;
```

```
    printf("Enter total number of disk blocks: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter disk request string:\n");
```

```
    for(i=0;i<n;i++)
```

```

{
    scanf("%d",&req[i]);
    visited[i]=0;
}

printf("Enter current head position: ");
scanf("%d",&head);

printf("\nSSTF Request Order:\n");
printf("%d",head);

for(i=0;i<n;i++)
{
    min=99999;
    pos=-1;

    /* Find nearest unvisited request */
    for(j=0;j<n;j++)
    {
        if(visited[j]==0)
        {
            diff=abs(head-req[j]);

            if(diff<min)
            {
                min=diff;
                pos=j;
            }
        }
    }
}

```

```

    }

    visited[pos]=1;

    total=total+abs(head-req[pos]);
    head=req[pos];

    printf(" -> %d",head);
}

printf("\n\nTotal Head Movements = %d\n",total);

return 0;
}

```

Q2)

```
#include<stdio.h>
```

```
struct frame
```

```
{
    int page;
    int counter;
};
```

```
int main()
```

```
{
    int ref[] = {3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
    int n = 15;
    int frame[20],counter[20];

```

```

int frames,i,j,pos,found;
int time=0,fault=0;

printf("Enter number of frames: ");
scanf("%d",&frames);

for(i=0;i<frames;i++)
{
    frame[i]=-1;
    counter[i]=0;
}

printf("\nPage\tFrames\t\tCounter\t\tFault\n");
printf("-----\n");

for(i=0;i<n;i++)
{
    time++;
    found=0;

    /* Check whether page is present */
    for(j=0;j<frames;j++)
    {
        if(frame[j]==ref[i])
        {
            found=1;
            counter[j]=time;
            break;
        }
    }
}

```

```
}
```

```
/* Page fault */
```

```
if(found==0)
```

```
{
```

```
    fault++;
```

```
    pos=-1;
```

```
/* Find empty frame */
```

```
for(j=0;j<frames;j++)
```

```
{
```

```
    if(frame[j]==-1)
```

```
    {
```

```
        pos=j;
```

```
        break;
```

```
    }
```

```
}
```

```
/* Find least recently used page */
```

```
if(pos== -1)
```

```
{
```

```
    pos=0;
```

```
    for(j=1;j<frames;j++)
```

```
    {
```

```
        if(counter[j]<counter[pos])
```

```
        pos=j;
```

```
    }
```

```
}
```

```

        frame[pos]=ref[i];
        counter[pos]=time;
    }

    printf("%d\t",ref[i]);

    for(j=0;j<frames;j++)
        printf("%d ",frame[j]);

    printf("\t");

    for(j=0;j<frames;j++)
        printf("%d ",counter[j]);

    if(found==0)
        printf("\tYES\n");
    else
        printf("\tNO\n");
}

printf("-----\n");
printf("Total Page Faults = %d\n",fault);

return 0;
}

```

Slip 28:-

Q.1 Write a simulation program for disk scheduling using SCAN algorithm.

Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

10, 25, 75, 90, 130, 145, 180, 55

Starting Head position= 80

Direction: Left

[15 Marks]

Q.2 Write the simulation program for demand paging and show the page

scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

[15 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int main()
```

```
{
```

```
    int blocks,n,i,j,temp;
```

```
    int req[100];
```

```
    int head,total=0;
```

```
    printf("Enter total number of disk blocks: ");
```

```
    scanf("%d",&blocks);
```

```
    printf("Enter number of disk requests: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter disk request string:\n");
```

```

for(i=0;i<n;i++)
    scanf("%d",&req[i]);

printf("Enter current head position: ");
scanf("%d",&head);

/* Sort requests */
for(i=0;i<n-1;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(req[i]>req[j])
        {
            temp=req[i];
            req[i]=req[j];
            req[j]=temp;
        }
    }
}

printf("\nSCAN Request Order:\n");
printf("%d",head);

/*
    Direction = Left
    First service requests smaller than head
*/

for(i=n-1;i>=0;i--)

```

```
{  
    if(req[i]<head)  
    {  
        total+=abs(head-req[i]);  
        head=req[i];  
  
        printf("-> %d",head);  
    }  
}
```

```
/* Move to beginning of disk */
```

```
if(head!=0)  
{  
    total+=head;  
    head=0;  
  
    printf("-> %d",head);  
}
```

```
/* Reverse direction and service remaining requests */
```

```
for(i=0;i<n;i++)  
{  
    if(req[i]>head)  
    {  
        total+=abs(head-req[i]);  
        head=req[i];  
  
        printf("-> %d",head);  
    }  
}
```

```

}

printf("\n\nTotal Head Movements = %d\n",total);

return 0;
}

```

Q2)

```

#include<stdio.h>

int main()
{
    int ref[] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
    int frame[20],freq[20];
    int n=15,frames;
    int i,j,pos,found;
    int faults=0;

    printf("Enter number of frames: ");
    scanf("%d",&frames);

    for(i=0;i<frames;i++)
    {
        frame[i]=-1;
        freq[i]=0;
    }

    printf("\nPage\tFrames\t\tFrequency\tFault\n");
    printf("-----\n");

```

```
for(i=0;i<n;i++)
{
    found=0;

    /* Check whether page is present */
    for(j=0;j<frames;j++)
    {
        if(frame[j]==ref[i])
        {
            found=1;
            freq[j]++;
            break;
        }
    }

    /* Page fault */
    if(found==0)
    {
        faults++;
        pos=-1;

        /* Find empty frame */
        for(j=0;j<frames;j++)
        {
            if(frame[j]==-1)
            {
                pos=j;
                break;
            }
        }
    }
}
```

```

    }
}

/* Find Most Frequently Used page */
if(pos==-1)
{
    pos=0;

    for(j=1;j<frames;j++)
    {
        if(freq[j]>freq[pos])
            pos=j;
    }
}

frame[pos]=ref[i];
freq[pos]=1;
}

printf("%d\t",ref[i]);

for(j=0;j<frames;j++)
    printf("%d ",frame[j]);

printf("\t\t");

for(j=0;j<frames;j++)
    printf("%d ",freq[j]);

```

```
    if(found==0)
        printf("\tYES\n");
    else
        printf("\tNO\n");
}

printf("-----\n");
printf("Total Page Faults = %d\n",faults);

return 0;
}
```

Slip 29:-

Q.1 Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array [15 Marks]

Q.2 Write the program to simulate FCFS CPU-scheduling. The arrival time and first CPU- burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15 Marks]

Q.3. Oral/Viva [5 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/types.h>
```

```
#include<sys/wait.h>
```

```
void sort(int a[],int n)
```

```
{
```

```
    int i,j,temp;
```

```
    for(i=0;i<n-1;i++)
```

```
        for(j=0;j<n-i-1;j++)
```

```
            if(a[j]>a[j+1])
```

```
            {
```

```
                temp=a[j];
```

```
        a[j]=a[j+1];  
        a[j+1]=temp;  
    }  
}
```

```
int main()  
{  
    int a[20],n,i,item;  
    char *args[25];  
    pid_t pid;  
  
    printf("Enter number of integers: ");  
    scanf("%d",&n);  
  
    printf("Enter integers:\n");  
  
    for(i=0;i<n;i++)  
        scanf("%d",&a[i]);  
  
    printf("Enter element to search: ");  
    scanf("%d",&item);  
  
    pid=fork();  
  
    if(pid<0)  
    {  
        printf("Fork Failed\n");  
        return 1;  
    }
```

```
if(pid==0)
{
    /* Child sorts the array */
    sort(a,n);

    printf("\nSorted Array: ");

    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    printf("\n");

    args[0]="./search";

    for(i=0;i<n;i++)
    {
        args[i+1]=malloc(12);
        sprintf(args[i+1],"%d",a[i]);
    }

    args[n+1]=malloc(12);
    sprintf(args[n+1],"%d",item);

    args[n+2]=NULL;

    execve("./search",args,NULL);

    printf("execve failed\n");
}
```

```
        exit(1);
    }
    else
    {
        wait(NULL);
        printf("\nParent process completed.\n");
    }

    return 0;
}
```

Q2)

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
```

```
struct process
```

```
{
    int pid;
    int at;
    int bt1;
    int bt2;
    int wt;
    int tat;
    int ct;
};
```

```
int main()
```

```

{
    struct process p[20],temp;
    int n,i,j,time=0;
    float awt=0,atat=0;

    srand(time(NULL));

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        p[i].pid=i+1;

        printf("\nEnter Arrival Time for P%d: ",i+1);
        scanf("%d",&p[i].at);

        printf("Enter First CPU Burst for P%d: ",i+1);
        scanf("%d",&p[i].bt1);

        /* Generate next CPU burst randomly */
        p[i].bt2=1+rand()%9;
    }

    /* Sort processes according to arrival time */
    for(i=0;i<n-1;i++)
    {
        for(j=i+1;j<n;j++)
        {

```

```

        if(p[i].at>p[j].at)
        {
            temp=p[i];
            p[i]=p[j];
            p[j]=temp;
        }
    }
}

printf("\nGenerated Next CPU Bursts:\n");

for(i=0;i<n;i++)
    printf("P%d = %d\n",p[i].pid,p[i].bt2);

printf("\nGantt Chart:\n");

for(i=0;i<n;i++)
{
    /* CPU idle if process has not arrived */
    if(time<p[i].at)
        time=p[i].at;

    p[i].wt=time-p[i].at;

    printf(" | P%d ",p[i].pid);

    /* First CPU burst */
    time+=p[i].bt1;

```

```

/* Next randomly generated CPU burst */
time+=p[i].bt2;

p[i].ct=time;
p[i].tat=p[i].ct-p[i].at;

awt+=p[i].wt;
atat+=p[i].tat;
}

printf("|\\n");

printf("\\nProcess\\tAT\\tBT1\\tBT2\\tWT\\tTAT\\n");

for(i=0;i<n;i++)
{
    printf("P%d\\t%d\\t%d\\t%d\\t%d\\t%d\\n",
        p[i].pid,
        p[i].at,
        p[i].bt1,
        p[i].bt2,
        p[i].wt,
        p[i].tat);
}

printf("\\nAverage Waiting Time = %.2f",awt/n);
printf("\\nAverage Turnaround Time = %.2f\\n",atat/n);
return 0;
}

```

Slip 30:-

Q.1 Write the program to simulate Round Robin (RR) CPU-scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. Assume the fixed IO waiting time (2 units). The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time

[15 Marks]

Q.2 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames.

Reference String: 2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8

[15 Marks]

Q.3 Oral/Viva

[05 Marks]

Q1)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<time.h>
```

```
struct process
```

```
{
```

```
    int pid,at,bt1,bt2,rem;
```

```
    int first_done,io,done;
```

```
    int ct,wt,tat;
```

```
};
```

```
int main()
```

```
{
```

```
    struct process p[20];
```

```
    int q[1000];
```

```
int n,quantum;
int front=0,rear=0;
int time=0,completed=0;
int current=-1,qtime=0;
int i,j;
int inqueue[20]={0};
float awt=0,atat=0;

srand(time(NULL));

printf("Enter number of processes: ");
scanf("%d",&n);

printf("Enter time quantum: ");
scanf("%d",&quantum);

for(i=0;i<n;i++)
{
    p[i].pid=i+1;

    printf("\nEnter Arrival Time for P%d: ",i+1);
    scanf("%d",&p[i].at);

    printf("Enter First CPU Burst for P%d: ",i+1);
    scanf("%d",&p[i].bt1);

    /* Generate next CPU burst randomly */
    p[i].bt2=1+rand()%9;
```

```

    p[i].rem=p[i].bt1;
    p[i].first_done=0;
    p[i].io=0;
    p[i].done=0;
}

printf("\nGenerated Next CPU Bursts:\n");

for(i=0;i<n;i++)
    printf("P%d = %d\n",p[i].pid,p[i].bt2);

printf("\nGantt Chart:\n");

while(completed<n)
{
    /*
     * Add processes which have arrived.
     */
    for(i=0;i<n;i++)
    {
        if(!p[i].done &&
            p[i].io==0 &&
            p[i].at<=time &&
            !inqueue[i] &&
            current!=i)
        {
            q[rear++]=i;
            inqueue[i]=1;
        }
    }
}

```

```
}
```

```
/* Select next process */
```

```
if(current==-1 && front<rear)
```

```
{
```

```
    current=q[front++];
```

```
    inqueue[current]=0;
```

```
    qtime=0;
```

```
    printf(" | P%d ",p[current].pid);
```

```
}
```

```
/* CPU idle */
```

```
if(current==-1)
```

```
{
```

```
    time++;
```

```
/* One unit of I/O time passes */
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    if(p[i].io>0)
```

```
    {
```

```
        p[i].io--;
```

```
        if(p[i].io==0)
```

```
            p[i].rem=p[i].bt2;
```

```
    }
```

```
}
```

```

        continue;
    }

    /* Execute one CPU unit */
    p[current].rem--;
    qtime++;
    time++;

    /* One unit of I/O time passes */
    for(i=0;i<n;i++)
    {
        if(p[i].io>0)
        {
            p[i].io--;

            if(p[i].io==0)
                p[i].rem=p[i].bt2;
        }
    }

    /* First CPU burst completed */
    if(p[current].rem==0 &&
       p[current].first_done==0)
    {
        p[current].first_done=1;
        p[current].io=2;

        current=-1;
        qtime=0;
    }

```

```
}
```

```
/* Second CPU burst completed */
```

```
else if(p[current].rem==0 &&  
        p[current].first_done==1)
```

```
{
```

```
    p[current].done=1;
```

```
    p[current].ct=time;
```

```
    p[current].tat=
```

```
        p[current].ct-p[current].at;
```

```
    p[current].wt=
```

```
        p[current].tat-
```

```
        p[current].bt1-
```

```
        p[current].bt2-2;
```

```
    awt+=p[current].wt;
```

```
    atat+=p[current].tat;
```

```
    completed++;
```

```
    current=-1;
```

```
    qtime=0;
```

```
}
```

```
/* Time quantum expired */
```

```
else if(qtime==quantum)
```

```
{
```

```

    q[rear++]=current;
    inqueue[current]=1;

    current=-1;
    qtime=0;
}
}

printf("|\\n");

printf("\\nProcess\\tAT\\tBT1\\tBT2\\tWT\\tTAT\\n");

for(i=0;i<n;i++)
{
    printf("P%d\\t%d\\t%d\\t%d\\t%d\\t%d\\n",
        p[i].pid,
        p[i].at,
        p[i].bt1,
        p[i].bt2,
        p[i].wt,
        p[i].tat);
}

printf("\\nAverage Waiting Time = %.2f",
    awt/n);

printf("\\nAverage Turnaround Time = %.2f\\n",
    atat/n);

```

```
    return 0;
}
```

Q2)

```
#include<stdio.h>
```

```
int main()
{
    int ref[] = {2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8};
    int n=16;
    int frame[20],freq[20];
    int frames,i,j,pos,found;
    int faults=0;

    printf("Enter number of frames: ");
    scanf("%d",&frames);

    for(i=0;i<frames;i++)
    {
        frame[i]=-1;
        freq[i]=0;
    }

    printf("\nPage\tFrames\t\tFrequency\tFault\n");
    printf("-----\n");

    for(i=0;i<n;i++)
    {
        found=0;
```

```
/* Check whether page is present */
```

```
for(j=0;j<frames;j++)
```

```
{
```

```
    if(frame[j]==ref[i])
```

```
    {
```

```
        found=1;
```

```
        freq[j]++;
```

```
        break;
```

```
    }
```

```
}
```

```
/* Page fault */
```

```
if(found==0)
```

```
{
```

```
    faults++;
```

```
    pos=-1;
```

```
/* Find empty frame */
```

```
for(j=0;j<frames;j++)
```

```
{
```

```
    if(frame[j]==-1)
```

```
    {
```

```
        pos=j;
```

```
        break;
```

```
    }
```

```
}
```

```
/* Find Most Frequently Used page */
```

```
if(pos==-1)
{
    pos=0;

    for(j=1;j<frames;j++)
    {
        if(freq[j]>freq[pos])
            pos=j;
    }
}
```

```
frame[pos]=ref[i];
freq[pos]=1;
}
```

```
printf("%d\t",ref[i]);
```

```
for(j=0;j<frames;j++)
    printf("%d ",frame[j]);
```

```
printf("\t\t");
```

```
for(j=0;j<frames;j++)
    printf("%d ",freq[j]);
```

```
if(found==0)
    printf("\tYES\n");
else
    printf("\tNO\n");
```

```
}
```

```
printf("-----\n");
```

```
printf("Total Page Faults = %d\n", faults);
```

```
return 0;
```

```
}
```